Table of Contents

BOOKLY **3**

FEJLESZTÉSI FOLYAMAT: 5

Csapatmunka: **5**

Forrásjegyzék: **5**

FRONTEND: 7

BACKEND: 7

ADATBÁZIS SZERKEZETE: 8

Motor: **8**

Táblák: **8**

Táblák közötti kapcsolatok: **10**

Adatbázis tervezési kikötések: **10**

CI/CD PIPLINE: 12

Leírás: **12**

Deployment folyamat: **12**

BACKEND: 13

AUTHENTIKÁCIÓ ÉS JOGOSULTSÁG KEZELÉS SZEREPKÖR ALAPJÁN: 13

Miért ez? *13*

Implementáció: *13*

Megújítási logika: *13*

Authentikációs Middlewarek: *14*

API Végpontok: *14*

MEGOSZTOTT HÁTTÉR-SZOLGÁLTATÁSOK ÉS SEGÉDFÜGGVÉNYEK: 16

Leírás: **16**

Backend: **16**

Frontend: **17**

Email service: *17*

Leírás: **17**

Érintett fájlok: **17**

Működési logika: **17**

ADMIN MODUL: 17

Bejelentkezési adatok: **17**

Leírás: **18**

Érintett fájlok: **18**

Működési logika: **18**

FELHASZNÁLÓI(CUSTOMER) MODUL: 21

Keresés és felfedezés: *21*

Érintett adatbázis táblák: **21**

API végpontok: **21**

Keresés: **21**

Ajánlott szalonok: **21**

Térkép integráció: **22**

FeaturedSalonsTab.jsx: **22**

Foglalásaim *22*

AppointmentsTab.jsx **23**

Mentett szalonok *23*

Leírás: **23**

Érintett fájlok és Adatbázis táblák: **23**

API végpontok: **24**

Működési logika: **24**

Üzenetek *24*

Leírás: **24**

Érintett fájlok és adatbázis táblák: **24**

API végpontok: **25**

Működési logika: **25**

Foglalási folyamat(booking-flow) *26*

Leírás: **26**

Érintett fájlok és adatbázis táblák: **26**

API végpontok: **27**
Működési logika: **27**
Szolgáltatási Belépési Pont (User Journey) *29*
Regisztráció és Bejelentkezés (Provider Onboarding) *29*
Leírás: **29**
Érintett fájlok és Adatbázis táblák: **29**
API végpontok (backend/api/auth/ProvLoginApi.js): **30**
Működési logika és Adatfolyam **31**
Irányítópult (Dashboard) – Fő nézet és Navigáció *33*
Leírás: **33**
Érintett fájlok és Adatbázis táblák: **33**
Működési logika és Adatfolyam **35**
Szalon Profil és Arculat Menedzsment *36*
Leírás: **36**
Érintett fájlok és Adatbázis táblák: **36**
API végpontok (backend/api/salon/settingsRoutes.js): **37**
Működési logika és Adatfolyam: **37**
Alkalmazottak és Csapat Kezelése (Staff Management) *38*
Leírás: **38**
Érintett fájlok és Adatbázis táblák: **39**
API végpontok (backend/api/salon/staffRoutes.js és backend/api/staffApi.js): **40**
Működési logika és Adatfolyam: **41**
Szolgáltatások és Árazás (Services CRUD) Kezelése *42*
Leírás: **42**
Érintett fájlok és Adatbázis táblák: **42**
API végpontok (backend/api/calendar/serviceRoutes.js): **43**
Működési logika és Adatfolyam: **44**
Naptár és Elérhetőség (Calendar & Availability) Kezelése *45*
Leírás: **45**
Érintett fájlok és Adatbázis táblák: **45**
API végpontok (backend/api/calendar/dashboardRoutes.js és backend/api/calendar/appointmentRoutes.js és backend/api/timeBlocksApi.js): **46**
Működési logika és Adatfolyam: **48**
Vendégkör kezelése (Customer Management) *50*
Leírás: **50**
Érintett fájlok és Adatbázis táblák: **50**
API végpontok (backend/api/calendar/customerRoutes.js): **51**
Működési logika és Adatfolyam: **52**
Üzenetek és Kapcsolattartás (Messaging / Chat) *53*
Leírás: **53**
Érintett fájlok és Adatbázis táblák: **53**
API végpontok (backend/api/messagingApi.js): **54**
Működési logika és Adatfolyam: **55**
Személyes Profil és Jelszókezelés (Provider Profile) *56*
Leírás: **56**
Érintett fájlok és Adatbázis táblák: **56**
API végpontok (backend/api/salon/providerProfileRoutes.js): **57**
Működési logika és Adatfolyam: **57**
FRONTEND TESZTELÉSI RENDSZER 59
Leírás **59**
Érintett fájlok és struktúra **59**
Tesztek felépítése és működési logika **61**
Screenshot mechanizmus **62**
Helper és Fixture architektúra **63**
Működési logika és teszt adatfolyam **63**
Összegzés **64**
Backend Tesztelési Rendszer *66*
Leírás **66**
Érintett fájlok és struktúra **66**
Tesztek felépítése és működési logika **67**
Riport és coverage mechanizmus **69**
Helper architektúra **70**
Működési logika és teszt adatfolyam **71**

Összegzés **71**
FELHASZNÁLÓI DOKUMENTÁCIÓ **73**
1. BEVEZETÉS 73
2. A RENDSZER FŐ FUNKCIÓI 73
3. A RENDSZER HASZNÁLATA – VENDÉG (CUSTOMER) 75
4. A RENDSZER HASZNÁLATA – SZOLGÁLTATÓ (PROVIDER) 78
5. ADMIN FUNKCIÓK 80

Bookly

Bookly-ról röviden:

A weboldal alap koncepciója egy olyan oldal megteremtése volt, ahol a felhasználók egyszerűen és könnyedén tudnak időpontfoglalásokat intézni különböző cégekhez, akik különböző szolgáltatásokat biztosítanak. Emellett a cégek, vagy más néven szolgáltatók egy egyszerű és jól átlátható felületen tudják kezelni a hozzájuk beérkező foglalásokat, és az ehhez kapcsolódó egyéb funkciókat. Ennek jegyében született meg a Bookly, ami ezekre a problémákra kínál megoldást. A célunk a Bookly-val az volt, hogy minél szélesebb körben nyújtson megoldást az időpont foglalás problémájára. Sokszor szembesülünk vele, hogy egy vállalkozó, aki valamilyen szolgáltatást nyújt (Pl.: egy barbershop ahova fodrászhoz járunk) egy rosszul működő időpont foglalási weboldalt használ. A Bookly-val ezek a problémák egyszerűen megoldhatóak, így nem kell a vállalkozónak egy rosszul működő átláthatatlan felületet használnia, és a papír alapú időpontfoglalást is elfelejtheti. Mi megoldást adunk a problémáira, így tud más dolgokra fokuszálni, nem kell az időpontokkal felesleges időt eltöltenie. Ezek mellett az oldalt, egy gyűjtő helyként is funkcionál, ahol a felhasználók tudnak böngészni, és új vállalkozásokat, szolgáltatókat felfedezni, és hozzájuk foglalást leadni. Tehát a Bookly egy modern, full-stack webes alkalmazás időpontfoglaláshoz, amely két fő felhasználói csoportot szolgál ki:

Ügyfelek (későbbiekben: Customer):
- Szolgáltatások böngészése és keresése
- Idöpontfoglalás egyszerü felületen
- Foglalások kezelése és nyomon követése
- Mentett szolgaltatok listaja

Szolgáltatok (későbbiekben: Provider):
- Uzleti dashboard komplex elemzésekkel

- Naptár kezelés és időpontok nyomon követése
- Szolgaltatasok menedzsmentje
- Ugyfél kapcsolattartás

Fejlesztési folyamat:

Csapatmunka:

A csapatmunka a Bookly megvalósítása során a csapat tagjai által elég jól megvalósult. Úgy gondoltuk, hogy a mivel egyikünk sem akart csak frontendel, vagy csak backendel foglalkozni, így úgy osztottuk fel a feladatot, hogy a weboldal egyik felét, azaz a Customer oldalt Varga Botond, a Provider oldalt pedig Tokaji-Nagy Bence valósította meg. Az admin oldalt is Varga Botond csinálta, mivel a Provider oldal önmagában egy komplexebb logikát, és több funkciót tartalmaz, mint a Customer oldal, így Tokaji-Nagy Bencére ott nagyobb feladat hárult. Emiatt a csapat mindkét tagja foglalkozott a frontend kinézettel, backend API végpontokkal, és adatbázis struktúra felépítéssel. A funkciók nagy részét közösen találtuk ki, és a fejelsztés során mindig kommunikáltuk egymásnak az éppen aktuális feladatainak, hogy mit fogunk fejleszteni a következő napokban / hetekben. Persze, a két rész között volt átfedés, mert néha úgymond belenyúltunk egymás részébe, tehát Varga Botond is fejlesztett a Provider oldalon, mint ahogy Tokaji-Nagy Bence is a Customer oldalon. A Foglalás mechanikájának megoldása teljes mértékben közös feladat volt, és ezt Discord hívások keretében valósítottuk meg. Viszont voltak olyan részek is, amik csak az egyik, vagy csak a másik tagját érintették a csapatnak.

Tokaji-Nagy Bence: A weboldal authentikációját és jogosultságkezelését tervezte meg, és alakította ki. A JWT access és refresh tokenek használatát teljes mértékben ő implementálta a kódba. A React és tailwind struktúrát is ő alakította ki. CI/CD folyamatot is ő tervezte meg amely github actions segítségével a repository-hoz társított szerverre telepíti az alkalmazás és futtatja azt.

Varga Botond: A kezdeti tervekben nagyobb szerepet játszott. A weboldal kezdeti kinézetét megtervezte, és finom hangolta. Az elképzelés, hogy hogyan nézzen ki a kezdő oldal itt keződött. Az adatbázis kezdeti struktúráját is ő tervezte meg, a drawSQL.app weboldalon. A drawio.com nevű oldalon tervezett egy feature map-et is az oldalhoz, ami segítségként szolgált arra, hogy melyik lapon milyen funkciók vannak, és egy szép képet kaptunk arról, hogy nagyjából milyen fejlesztési feladataink vannak.

Forrásjegyzék:

2025.10 – 2026.01 – React tanulása különböző kurzusokból -
https://www.youtube.com/watch?v=CgkZ7MvWUAA
https://richardkorom.hu/javascript/react/bevezetes/
https://www.udemy.com/course/react-the-complete-guide-incl-redux/?src=sac&kw=react&couponCode=KEEPLEARNING

2025.11.07 – Jelszó titkosítás, BCRYPT használata -
https://www.youtube.com/watch?v=qgpsIBLvrGY

2025.11.08 – jelszó titkosítás -
https://www.youtube.com/watch?v=AzA_LTDoFqY

2025.11.23 – DOTENV használata -
https://www.youtube.com/watch?v=AcYF18oGn6Y

2025.12.09 – JWT webtoken használata és megismerése -
https://www.youtube.com/watch?v=Y2H3DXDeS3Q
https://www.youtube.com/watch?v=AcYF18oGn6Y

2025.12.10 – Cookies megismerése -
https://www.youtube.com/watch?v=s04Vjlcgwco
https://www.youtube.com/watch?v=rdVPflECed8

2026.02.24 – Leaflet és Maptiler használata -
https://www.youtube.com/watch?v=wVnimcQsuwk&list=PLGHe6Moaz52PUNP4DtIshALDogSURIlYB

Tech Stack:

Frontend:

Framework: React

Framework-nek React került kiválasztásra mivel egyszerű “Single- page” weblapokat lehet vele létrehozni gyors és folyékony UI-al, mindezek mellett megkönnyíti a fejlesztés menetét mivel sima html+JavaScripttel ellentétben deklaratív módon történik a fejlesztés.

Styling: Tailwindcss

Az oldal komponenseinek designolásához Tailwind-et használtunk mivel ez hatékonyabban együtt működik a React render mechanikájával.

Backend:

Runtime: Node.js

Framework: Express

Backend Server felépítéséhez express-t használtunk esze lehetővé téve hogy az egész applikáció JavaScript segítségével legyen fejlesztve.

Database:MySQL

Adatbázis felépítéséhez MySQL-t használtunk, mivel a MySQL egy relációs adatbázis így átláthatóságot biztosít, valamint szinte már “Industry-Standard” -nek lehet nevezni.

Layer	Technology
Frontend	React 19, Vite 7, Tailwind CSS 4, React Router 7, Leaflet (maps), react-datepicker
Backend	Express 5 on Node.js, mysql2 (promise API, connection pool), JWT auth, Sharp, Multer
Database	MySQL 8.0 in Docker

Layer	Technology
Auth	JWT access tokens (15min) + refresh tokens (7d) as HttpOnly cookies, bcryptjs
Real-time	Socket.IO élő beszélgetéshez
Email	nodemailer via Gmail SMTP

Adatbázis szerkezete:

Motor:

MySQL 8, utf8mb4_hungarian_ci karakterkészlettel, Europe/Budapest időzónával. Docker konténerben fut (bookly-db), az init.sql az első indításkor automatikusan létrehozza a sémát és feltölti a demo adatokat.

Táblák:

Tábla	Leírás
admins	Rendszeradminisztrátorok (külön tábla a biztonság érdekében)
users	Ügyfelek – státusz: active/inactive/deleted/banned, szerepkör: user/employee/customer
providers	Szolgáltatók – szalonhoz kötve (salon_id), szerepkör: provider/manager, isManager flag
salons	Szalonok – cím, nyitvatartás, geokódolt koordináták, sharecode, banner/logo URL
services	Szolgáltatások – egy providerhez tartoznak, ár, időtartam (perc), státusz
appointments	Foglalások – user/provider/service kapcsolattal, vendég-adatokkal (regisztráció nélküli foglaláshoz), soft delete
ratings	Értékelések – foglaláshoz kötve (1 foglalás = max 1 értékelés), szalon + provider külön 1–5 csillag
saved_salons	Kedvenc szalonok (user ↔ salon, unique pár)
provider_time_blocks	Időblokkok (pl. ebéd, szabadság) – ismétlődő (daily/weekly) vagy egyszeri
waitlist	Várólista – dátum- és időtartomány, státusz: active/booked/expired/canceled
conversations	Üzenetszálak (provider ↔ user egyedi pár), olvasatlan számlálóval
messages	Üzenetek – küldő szerepkörrel, opcionális foglalás-hivatkozással
service_images	Szolgáltatás-képek (több kép/szolgáltatás, sorrendezéssel)

RefTokens	Refresh tokenek – user/provider/admin bármelyikéhez köthető
system_logs	Audit napló – szint: INFO/WARN/CRITICAL, akció, aktor, cél

Táblák közötti kapcsolatok:

Szalon és munkatársak
Minden salon-hoz több provider tartozhat. A providerek közül egy vagy több isManager = TRUE flaggel rendelkezik, ők kezelik a szalont.

Providerek és szolgáltatások
Minden provider saját service-eket (szolgáltatásokat) kínál. Egy szolgáltatáshoz több kép is feltölthető (service_images), sorrendezéssel.

Foglalások
Az appointment mindig egy provider-hez és egy service-hez kötött. A foglalást leadó fél lehet regisztrált user (ilyenkor user_id töltött), vagy vendég (ilyenkor user_id = NULL, és a guest_name/email/phone mezők tartalmazzák az adatokat).

Értékelések
Egy lezárt appointment-hoz pontosan egy rating adható le. Az értékelés egyszerre vonatkozik a szalonra és a providerre is (külön 1–5 csillag és szöveges vélemény).

Időblokkok és várólista
Minden provider rögzíthet provider_time_blocks bejegyzéseket (pl. ebédszünet, szabadság), amelyek egyszeri vagy ismétlődő (napi/heti) blokkok lehetnek. Ha nincs szabad időpont, a user feliratkozhat a waitlist-re egy adott providernél, megadva a kívánt dátum- és időtartományt.

Kedvenc szalonok
A user tetszőleges számú salon-t elmenthet magának. Egy user ugyanazt a szalont csak egyszer mentheti el (unique kényszer).

Üzenetváltás
Egy user és egy provider között pontosan egy conversation létezhet. A conversation-hoz tartoznak a konkrét message bejegyzések, amelyek hivatkozhatnak egy appointment-ra is (pl. foglalással kapcsolatos üzenetek esetén).

Hitelesítés
A RefTokens tábla tárolja a refresh tokeneket. Egy sor mindig pontosan egy entitáshoz – user, provider vagy admin – köthető (a másik két foreign key ilyenkor NULL). Fiók törlésekor a tokenek automatikusan törlődnek (cascade).

Audit napló
A system_logs minden fontos eseményt rögzít (pl. LOGIN, REGISTER, BAN, GDPR_DELETE). Az actor_type megmondja ki hajtotta végre (admin, user, provider vagy system), a target_type/target_id pedig hogy melyik rekordra vonatkozott.

Adatbázis tervezési kikötések:

Soft delete mindenhol – rekordok soha nem törlődnek fizikailag. Az appointments, users és providers táblákon deleted_at timestamp és status = 'deleted' jelzi a törölt állapotot, így a historikus adatok és audit trail megmarad.

Vendégfoglalás – az appointments táblában a user_id szándékosan nullable. Regisztrált felhasználó nélküli foglalásnál a guest_name, guest_email és guest_phone mezők veszik át a szerepét.
Admin elkülönítése – az adminok külön admins táblában vannak, nem a users-ben. Ez megakadályozza, hogy egy user véletlenül vagy szándékosan admin jogosultságot kapjon.

Refresh tokenek rugalmassága – a RefTokens tábla egyetlen sorban tudja kezelni a három különböző entitástípust (user, provider, admin) úgy, hogy mindig csak az egyik foreign key töltött, a másik kettő NULL. Ezzel elkerülhető három külön token-tábla karbantartása.

Ismétlődő időblokkok – a provider_time_blocks tábla is_recurring, recurrence_pattern (daily/weekly), recurrence_days (JSON) és recurrence_end_date mezőkkel kezeli az ismétlődő blokkokat anélkül, hogy minden napra külön sort kellene felvenni.

Üzenetváltás unique kényszere – a conversations táblán UNIQUE KEY unique_conversation (provider_id, user_id) biztosítja, hogy egy provider és egy user között csak egyetlen szál létezhet, duplikált szálak nem keletkezhetnek.

Értékelés és foglalás 1:1 kapcsolata – a ratings táblán UNIQUE KEY unique_appointment_rating (appointment_id) garantálja, hogy egy foglaláshoz legfeljebb egy értékelés adható le.

CI/CD Pipline:

Leírás:

A program github actions segítségével valósítja meg a CI/CD folyamatot. Minden mainbe történő push után lefut a deployment folyamat. A szerver adatait a repository secret tárhelyében tároljuk és a script onnan tölti be őket, így ezen adatok cseréjével akár másik szerverre is fel lehet tölteni a programot. A script feltételezi, hogy a szerver elő van készítve a program futtatására Pl.: Node, Docker, Nginx telepítve és konfigurálva van. Jelen esetben ez a CI/CD folyamat productionra NEM alkalmas mivel nem tartalmaz tesztelést a deployment előtt így hibás kód is felkerülhet a szerverre, valamint nem rendelkezik database migrationnal illetve rollback lehetőséggel, így ez rendszer csak fejlesztésre alkalmas. A deploy folyamatot deploy.yml fájlban tárolja a program

Deployment folyamat:

- **Kapcsoloódás a szerverhez ssh kulcs segítségével:** a github secret között eltárolt Private kulcs segítségével csatlakozik a szerverhez.
- **Repository klónozása:** a régi már meglévő helyi fájlok törlésre kerülnek és újra klónozásra a repositoryból.
- **Környezeti változók betöltése:** a szerveren permanensen jelen lévő környezeti változói fájlt (.env) bemásolásra kerül a program backend mappájába.
- **Adatbázis újra indítása:** az adatbázis docekr-compose down-v majd docker-compose up -d db parancsokkal újra létrejön
- **Backend előkészítés:** backenden lefut az npm install valamint pm2 telepítésre kerül ha még nincs telepítve a szerverre.
- **Frontend előkészítés:** npm install illetve npm run build paranccsal létrehozásra kerül a frontend fájlokat tartalmazó /dist mappa
- **Nginx deployment:** a /dist mappa tartalma áthelyezésre kerül az Nginx mappájába majd az Nginx service újraindításra kerül a systemctl reload nginx paranccsal.
- **Backend elindítása:** a backend elindításra kerül pm2 segítségével a pm2 start „npm start” –name bookly-backend paranccsal. Pm2 újra építi a backendet, ha az kifagyna.

Backend:

Authentikáció és jogosultság kezelés szerepkör alapján:

Az alkalmazás JWT (jsonwebtoken) alapú authentikációt használ Frissítési mechanizmussal, ez kiváltja a session használatát.

Miért ez?

- Szervernek nem kell semmit tárolnia
- Tökéletes RESTful elv betartásához mivel így minden kérés különálló és független
- Többféle kliens használathoz előnyös
- Token payload-ból DB lookup nélkül lehet adatokat kiolvasni

Implementáció:

- A felhasználók bejelentkezés után egy accessTokent illetve egy RefreshTokent kapnak. Az accessToken egy rövid élettartamú (15m) token melyet a frontrenden localStorageban tárolunk el, Refreshtoken egy hosszú élettartamú (7d) token melyet egy HTTPonly cookie-ban tárolunk ezzel védekezve az esteleges XSS támadások ellen.
- Ezek után a frontend kliens minden olyan api hívásához, ami védett, egy segéd fetch functiont (auth.js) használ, amely segít a (későbbiekben említett) authentikációs Middlewarek válaszait, illetve a token megújító logikát kezelni

Megújítási logika:

- Amikor a kliens API hívást kezdeményez egy védett végpont fele azt az “authApi” functionnal teszi ezután jön az AuthMiddleware amely ellenőrzi a token érvényességét. Érvényes token estén továbbítja a tokenből dekódolt adatokat a requesthez csatolva, Érvénytlene token esetén 401-es hibakóddal küld vissza hibát, amelyet az authFetch function kezel, először megpróbálja megújítani a tokent a refreshAccesToken

functionnal, sikeres megújítás esetén újrapróbálkozik a hívással, sikertelen megújítás esetén átirányítja a felhasználót a bejelentkezési felületre.

Refresh Queue logika:

- Ez megakadályozza, hogy egyszerre több hívás is megpróbálja frissíteni a tokent.
- Első hívás megpróbálja frissíteni a tokent
- Többi hívás addig vár
- Ha az első hívás elindított egy megújítást a többi csatlakozik egy refreshQueue-hoz
- Feliratkozás esetén hozzáad egy callback-et a queue hoz és sikeres token frissítés után meghívja a listában lévő callbackeket az új tokennel

Authentikációs Middlewarek:

- AuthMiddleware: ellenőrzi a token érvényességét.
- RoleMiddleware: ellenőrzi, hogy a token payloadbmn tárolt jogosultsági kör egyezik-e a paraméterekben megadottak valamelyikével.
- VerifyProvider: ellenőrzi, hogy a felhasználó létezik-e, illetve érvényes fiókkal rendelkezik-e.

API Végpontok:

/auth - Felhasználói bejelentkezés és hitelesítés

/auth/provider - Szolgáltatói (provider) bejelentkezés/hitelesítés

/api/search - Keresési funkciók (pl. szolgáltatás/szalon keresés)

/api/user - Felhasználói adatok és profil műveletek

/api/provider/calendar - Szolgáltatói naptár kezelése

/api/provider/time-blocks - Szolgáltatói időblokkok/idősávok kezelése

/api/salon - Szalon adatok és kapcsolódó műveletek

/api/admin - Adminisztrációs funkciók

/api/staff - Munkatársak (staff) kezelése

/api/messages - Üzenetküldés és üzenetek kezelése (Socket.IO támogatással)

/api/waitlist - Várólista kezelése

Cross Origin Resource Sharing (CORS):

CORS szabályozza, hogy a webes alkalmazások hogyan érhetnek el erőforrásokat különböző domain-ekről. Same-origin alkalmazásokkal ellentétben ebben a projektben a frontend kivitelezésének köszönhetően külön domainekről történik a Serveren lévő API-k hívása, ezért van szükség a CORS használatára.

CORS konfigurációjában beállításra került az engedélyezett domain-ek listája, valamint az érkező http csomagok struktúrája, illetve metódusa.

```
// CORS configuration - Allow all for development
app.use(cors({
    origin: ['http://localhost:5173', 'http://127.0.0.1:5173', 'http://localhost:5174', 'http://127.0.0.1:5174'],
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS', 'PATCH'],
    allowedHeaders: ['Content-Type', 'Authorization']
})); //?CORS middleware
```

Features:

Megosztott háttér-szolgáltatások és segédfüggvények:

Leírás:

A Provider modul számos végpontja közös háttér-rétegeket használ. Ezeket nem szekciónként, hanem itt összefoglalva dokumentáljuk, hogy ne ismétlődjenek.

Backend:

- **middleware/providerMiddleware.js:** verifyProvider (a calendar és time-blocks API-knál a request.providerId mezőt állítja be aktív (status='active') státusz-ellenőrzés után) és isManagerMiddleware (a manager-only szalon-végpontokon (PUT /api/salon/update, PUT /api/salon/status, POST /api/salon/branding, POST /api/salon/branding/remove-banner), valamint a csapat-módosító végpontokon (PUT/DELETE /api/salon/provider/:id); menedzseri jogkört ellenőriz, és a request.salonId mezőt a megerősített manager szalonjának azonosítójára állítja.)
- **middleware/uploadMiddleware.js:** Multer-alapú multipart feldolgozás, valamint Sharp-alapú képfeldolgozó utak (processSalonLogo, processSalonBanner, processServiceImage, processAndSaveImage) és a kapcsolódó takarítás (deleteOldSalonImage, deleteOldImage). Minden Provider-oldali képfeltöltés ezt használja.
- **utils/authUtils.js:** generateProviderTokens (rövid életű access + hosszabb refresh JWT generálása), setAuthCookies (HttpOnly + Secure cookie-k beállítása).
- **utils/dateUtils.js:** formatLocalDatetime – a foglalás-időpontok MySQL-konform serializálásáért felelős, használatban a manuális foglalás végpontnál.
- **services/emailService.js:** sendWelcomeEmail, sendCustomerReminder, sendAppointmentCancellation – minden Provider-aktivált e-mailt itt fut keresztül.
- **services/availabilityService.js:** A getFullyBookedDays és a foglalási rés-számítás, amelyre a fully-booked-days végpont és a Customer foglalási felület is támaszkodik.
- **services/waitlistService.js:** notifyWaitlistForCancelledSlot – a DELETE /api/provider/calendar/appointments/:id indítja aszinkron módon (a hívás visszatérési értékét nem várja be a végpont; az értesítések sikertelensége nem okoz hibát az API-válaszban).
- **services/logService.js:** logEvent – a Provider modul minden írási műveletét naplózza a system_logs táblába (PROVIDER_LOGIN, PROVIDER_SIGNUP, PROVIDER_SALON_JOIN, SALON_CREATED, SALON_UPDATED, SALON_BRANDING_UPDATED, SALON_STATUS_UPDATED, SERVICE_CREATED/UPDATED/DELETED, PROVIDER_APPOINTMENT_CREATED (regisztrált felhasználóhoz kötött manuális foglalás), PROVIDER_APPOINTMENT_CREATED_GUEST (vendég-foglalás),

PROVIDER_APPOINTMENT_CANCELED, PROVIDER_TEAM_UPDATED, PROVIDER_TEAM_REMOVED, PROVIDER_PROFILE_UPDATED, PROVIDER_PASSWORD_CHANGED, TIME_BLOCK_CREATED, MESSAGE_SENT).

- **backend/api/auth/RoleMiddleware.js:** a requireRole([...]) factory, amely a JWT-ből feloldott role mező alapján gátolja meg a más szerepkörű kérők belépését (a calendar és time-blocks API-knál ['provider']-rel).

Frontend:

- **modules/auth/auth.js:** Az authApi fetch-wrapper (get/put/post/upload), amely automatikusan injektálja az Authorization: Bearer ... headert és kezeli a 401 → refresh-flow-t: a POST /auth/refresh (egységes, szerepkörtől független) végpontot hívja a HttpOnly cookie-ban tárolt refresh tokennel, sikeres frissítés esetén megismétli az eredeti kérést az új access tokennel, sikertelen frissítés esetén kilépteti a felhasználót és átirányítja a /ProvLogin útvonalra. A Provider modul gyakorlatilag minden hitelesített hívása ezt használja.
- **components/NotificationContext.jsx:** Globális Toast + Confirm rendszer (useNotification).
- **components/skeletons/:** Skeleton-loader komponenscsalád.
- **hooks/useDebounce.js:** Általános debounce hook.

Email service:

Leírás:

Az oldal a különböző értesítéseket email-ben küldi el, ezekhez a nodemailer package-et használja.

Érintett fájlok:

- emailService.js

Működési logika:
Az emailek elküldéséhez a nodemailer package-t használja a program, ehez először létrehozunk egy „transportert” ahova meg kell adni az smtp szerver adatait, valamint a hozzátartozó cím belépési adatait. Ebben a programban a Google smtp szolgáltatását használjuk.ez után ezt a transportert használva tudjuk elküldeni az emaileket amiknek a törzsét html kódként kell megadni.

Admin modul:

Bejelentkezési adatok:
Email: admin@bookly.com

Jelszó: admin123

Leírás:
A felhasználók és szolgáltatók adatait és az oldalon történő tevékenységet. Az oldalon egy oldalsó navigációs sáv segítségével tudunk a Nézetek között váltogatni.

Érintett fájlok:

- **Frontend:**
  - **Adminlogin.jsx:** bejelentkezési oldal
  - **AdminDashboard.jsx:** áttekintési oldal
  - **UserManagement.jsx:** felhasznló management oldal.
  - **ProviderManagement.jsx:** szolgáltató management
  - **SalonManagement.jsx:** szalonok kezelése
  - **AppointmentManagement.jsx:** Foglalások kezelése
  - **RatingManagement.jsx:** értékelések kezelése
  - **SystemLogs.jsx:** Log

Működési logika:

Azoldalon az összes API végpont a /api/admin route-on helyezkedik így később ez nem lesz említve.

Áttekintés: ez az első oldal betöltés után itt látható egy átfogó statisztika az oldal aktuális adatairól, amelyet **/api/admin/statistics** API-n keresztül kapunk meg.

Felhasználó management: ezen a felületen lehet kezelni a felhasználókat. A felhasználók egy listában jelennek meg egymás alatt és rájuk kattintva jellenek meg a további opciók. Az oldal tetején egy kereső mező található. A listaelemre kattintva a további opciók között a következőek találhatóak:

- **Felhasználó tiltása/tiltás feloldása:** a felhasználó tiltása után a rendszer nem engedi bejelentkezni az adott felhasználót. A folyamatokat a **/users/:id/ban** vagy -/**unban** végpontokkal kezeli.
- **Profilkép törlése:** kitörli a felhasználó profil képét a **/uploads** mappából a **/users/:id/picture** API segítségével.
- **Profil törlése:** soft delete történik tehát a felhasználó nem lesz eltávolítva az adatbázisból csak a érzékeny információk lesznek NULL értékre állítva.ez folyamat nem visszaállítható a profilvisszaállítás funkcióval. A folyamat a /**users/:id/gdpr-delete** API-n keresztül történik.
- **Foglalások és értékelések:** ki listázza a felhasználóhoz tartozó értékeléseket és foglalásokat, valamint azokat törlni lehet.

Szolgáltató management: ezen a felületen lehet kezelni a szolgáltatókat. A felhasználói részhez hasonlóan itt is egy listában jelennek meg a szolgáltatók és rájuk kattintva a további opciók:

- **Szolgáltató tiltása/tiltás feloldása:** tiltott szolgáltatók nem tudnak bejelentkezni. A folyamat **/providers/:id/deactivate** APi-n keresztül történik.

- **Szolgáltató deaktiválása:** deaktivált szolgáltatók be tudnak jelentkezni viszont a foglalásnál nem jelennek meg, mint elérhető szolgáltató. A deaktiválás a **/providers/:id/deactivate** API-n keresztül történik.
- **Profilkép törlése:** törölni lehet az adott szolgáltató profil képét. A folyamat a /providers.
- **Foglalások értékelések és szolgáltatások kezelése:** az adott szolgáltatóra kattintva válik elérhetővé ez a menü. Itt tudjuk kezelni a szolgáltatóhoz tartozó foglalásokat, értékeléseket és szolgáltatásokat kezelni.

Szalon management: ebben a menüpontban tudjuk kezelni a szalonokat. A szalonok a többi menüponthoz hasonlóan itt is egy listában jelennek meg, a listához tartozik egy kereső mező is. A szalonokra kattintva az alábbi opciók elérhetőek:

- **Státusz megváltoztatása:** a szalon státuszát „nyitva", "zárva" illetve „felújítás" ra lehet változtatni. A folyamat a /salons/:id/status végponton keresztül történik.
- **Banner és profilkép törlése:** banner és profil kép törlése a /salons/:id/banner illetve /logo Delete APIk-on keresztül történik.
- **Leírás törlése:** a szalon leírásának törlése a /salons/:id/description Delete API-n keresztül történik.

Időpont management: a foglalások egy listában jelennek meg, a listaelem tartalmazza a foglalás fő adatait. Az oldal tetején a kereső mező alatt megtalálható egy statisztika rész is. A /**appointments/:id** delete API-n keresztül lehet a foglalásokat törölni, a törlés előtt egy indoklást kell az adminisztrátornak megadnia, amely eltárolásra kerül az adatbázisban.

Értékelések kezelése: az értékelések listában jelennek meg, rájuk kattintva kerül megjelenítésre az értékelések üzenete. Az értékeléseket a **/ratings/:id** Delete API-n keresztül lehet deaktiválni.

System Logs: a program minden fontosabb eseményt logol ami az oldalon történik, ezek a logok a **system_logs** táblában kerülnek eltárolásra. Logolás központosítva van egy service-ben, amelyet a logService.js fájlban találunk.

A **logEvent** függvényaz alábbi paraméterket kéri:

- **Level:** ez az esemény fontosságát jelzi ez lehet INFO, WARN vagy CRITICAL, később ezeket a log listában különböző színnel látja el a gyors megkülönböztetés érdekében.
- **Action:** általános leírás az eseményről
- **Actor-type**: ki kezdeményezte Pl.: admin,system,user
- **Actor_id**: kezdeményező ID-je
- **Target_type**: célzott típusa (NULL érték, ha nincs konkrét cél)
- **Target_id**: célzott ID-je
- **Details**: az esemény részleteinek leírása
- **Created_at**: létrehozás dátuma

Események fontossági szintjei:

- **INFO:** általános események PL.: login, foglalás
- **WARN:** visszafordítható de lényeges változás PL.: tiltás, deaktiválás
- **CRITICAL:** visszafordíthatatlan események PL.: GDPR delete

Felhasználói(customer) modul:

Keresés és felfedezés:

Érintett adatbázis táblák:

- salons
- providers
- services
- ratings

API végpontok:

Method	Útvonal	Leírás
POST	/search/nearby	Hely alapú keresés
GET	/search/by-name	név alapú keresés
GET	/search/suggestions	keresési ajánlások (autocomplete)
GET	/search/top-rated	értékelés alapú keresés
GET	/search/recent-reviews	keresés legutolsó értékelés alapján
GET	/search/types	típus alapú keresés
GET	/search/salon/:id	szalon adatok visszatérítése
GET	/search/address-autocomplete	cím autocomplete
POST	/search/geocode	cím koordinátára alakítása
POST	/search/reverse-geocode	koordináta címre alakítása
GET	/search/recommendation	egyedi szalon ajánlások

Keresés:

A keresési mező a következő funkciókat tartalmazza:

- keresés név alapján (debouncing autocomplete)
- keresés típus alapján
- keresés helymeghatározás alapján
- keresés a térkép használatával

A keresés eredményét a térkép is mutatja, valamint megjelennek a keresési eredménynek megfelelő szalonok kártyái amire kattintva meglehet nyitni az adott szalon felületét.

Ajánlott szalonok:

az ajánlott szalonok listáját egy complex sql lekérdezés adja vissza mely több szempontot figyelembe véve és azok eredményét súlyozva adja vissza az eredményt. figyelembe veszi hogy a helyek ahova a felhasználó már foglalt oda kik foglaltak még és nekik milyen más

foglalásaik vannak (collaborative filtering), szalon népszerűségét, a felhasználó régebbi foglalásait valamint a távolságot.

Térkép integráció:

Az alapja a **BaseMap.jsx** ami a leaflet-nek a keretként szolgál és react memo-n keresztül megakadályozza a komponens újra renderelését. Leaflet egy MapTiler nevű API-t használ, amin keresztül kapja meg a térképadatokat. Később ezt a komponenst használja fel az összes többi térkép funkció alapúl.

SalonMap.jsx:

Az előző basemap komponenst felhasználva renderel egy térképet, amelyen megjeliníti a szalonokat kék jelölőkkel. Ezeket a szalonok hosszúság és szélesség koordinátáit használva számolja ki a haversine formulát használva. A felhasználó helyzetét a „helyzetem" gombra kattintva piros jelölővel jelöli a térképen. A térképen a keresések végeredményeként a beépített flyTo függvényt használva közelít rá az eredményre a térképen. Amikor aktív keresés van érvényben csak a keresés eredményeit mutatja a térképen.

Kiemelt szalonok:

FeaturedSalonsTab.jsx:

A kiemelt szalonokat a rendszer az értékelések alapján választja ki és rangsorolja.

Foglalásaim

Method	Útvonal	Leírás
Get	/api/user/appointments	visszaadja a felhasználó foglalsait
Get	/api/waitlist	Visszaadja felhasználó várólistáit
Delete	/api/waitlist/:id	Töröl egy kiválasztott várólistát
Delete	/api/user/appoinments/:id	Töröl egy időpontot
Patch	/api/user/appointments/:id/comment	Szerkeszti az időponthoz tartozó megjegyzést
Get	/api/user/provider/:id/availability?	Visszaadja egy szolgáltató elérhetőségét a

		paraméter szerinti idősávban

AppointmentsTab.jsx

A komponens betöltésekor API hívás történik, amely az appointments state be menti el a felhasználó foglalási adatait, ezek után szétválasztja ezeket múltbéli, illetve esedékes időpontokra majd megjeleníti őket ez alapján.

A naptár a generateCalendarDays metódust használva készíti el a naptár UI-t az aktuális hónap alapján majd kiemeli az aktuális napot, illetve azokat a napokat, ahol van esedékes foglalás egy jelölővel látja el.

A napi bontás a foglalások kezdő és végpontjai alapján történnek, ahol minden esemény annyi helyet foglal el arányosan amennyi időt igénybe vesz.

Az oldal alján a felhasználó kezelni tudja a várólistáit (ha van neki) (több: Várólista szekció).

A mér elvégzett foglalásokat felhasználó ezen a felületen keresztül tudja értékelni a **Ratingmodal** komponensen keresztül.

Ratingmodal.jsx

A komponens a loadExistingRating metódust használva a **/api/user/ratings/appointments/:id** végponton keresztül betölti a már létező értékelést, ha van. Ezt követően megjelenik a értékelési Form amit kitöltve és elmentve az értékelés bekerül az adatbázisba a **/api/user/ratings** végponton keresztül.

CustomerAppointmentModal.jsx

Ha felhasználó rákattint egy időpontra megjelenítésre kerül a modal amely kijelzi az foglalás összes adatát. Ebben a komponensben nem történik API hívás mivel az adatokat Prop-ként kapja meg a szülő komponenstől.

Mentett szalonok

Leírás:

Ezen a felületen keresztül tudják a felhasználók megtekinteni és kezelni a mentett szalonjaikat.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek:**
    - **SavedSalonsTab.jsx:** megjeleníti a felhasználó által mentett szalonokat
    - **SalonCard.jsx:** megjeleníti a szalon adatait
- **Adatbázis táblák:**
    - **Saved_salons**

API végpontok:

Method	Útvonal	Leírás
Get	/api/user/saved-salons	Visszaadja a mentett szalonokat
POST	/api/user/saved-salons/:id	Elhelyezi a szalont a mentett szalonok közé
DELETE	/api/user/saved-salons/:id	Kitörli a szalont a mentett szalonok közül

Működési logika:

A felhasználói felületen lévő mentés gombok API hívásokkal adják hozzá, illetve törlik ki a menttet szalonok adatbázisból az adott szalont, amit a **Dashboard.jsx** fájlban található **toggleSaveSalon** metódus irányít.

Üzenetek

Leírás:

A felhasználók és a szolgáltatók üzenetket küldhetnek egymásnak, ha van közelgő közös időpontjuk.
Mindezt Websocket-et használva valós időben tehetik meg. A már elindított beszélgetéseket az adatbázis tárolja így azokat bármikor folytatni lehet.

Érintett fájlok és adatbázis táblák:

- **Frontend komponensek:**
    - **MessagesTab.jsx:** ez a komponens kezeli a socket kapcsolatot, illetve a State-eket egyfajta elosztó a többi komponensnek
    - **MessageThread.jsx:** rendereli a beszélgetést
    - **ConversationList.jsx:** rendereli a beszélgetéseket
    - **messagingService.js:** ebben a fájlban vannak az API hívásokhoz használt függvények
- **Adatbázis Táblák:**
    - **Conversations:**
        - **Provider/user Id:** a két résztvevő
        - **Last_message_preview:** a legutolsó üzenet
        - **Last_message_at:** utolsó üzenet ideje
        - **Provider/user_unread_count:** meg nem tekintett üzenetek száma
    - **Messages:**
        - **Sender_role:** a küldő szerepköre

- **Sender_id:** idegen kulcs a küldőhöz
- **Content:** üzenet
- **Appointment_id:** idegen kulcs a beszélgetéshez tartozó időponthoz

API végpontok:

Method	Útvonal	Leírás
GET	/api/messages/conversations	Visszaadja az összes beszélgetésést a bejelentkezett felhasználónak
GET	/api/messages/conversations/:id	Visszaadja a teljes beszélgetés id alapján
POST	/api/messages/conversations	Új beszélgetés létrehozása
POST	/api/messages/conversations/:id/messages	Üzenet küldése
PATCH	/api/messages/conversations/:id/read	Megjelöli az üzenetet nézettként
DELETE	/api/messages/conversations/:id	Kitörli a beszélgetést
GET	/api/user/appointments	Segít visszaadni azt a listát, hogy kivel tud beszélgetést indítani a felhasználó

Működési logika:

- **Beszélgetés indítása:** a felhasználónak az új beszélgetés gombra kattintva feldob egy menüt, amelyben azok a szolgáltatók szerepelnek, akikkel már volt vagy lesz közös foglalásuk. A gombra kattintva meghívásra kerül a **loadEligibleProviders** függvény, amely visszaadja azon szolgáltatók listáját, akikkel beszélgetést tudunk kezdeményezni. Ezután meghívásra kerül a **handleStartConversation** függvény, amely API híváson keresztül létrehoz egy új beszélgetést majd megjeleníti azt.

- **Beszélgetés lista betöltése:** amikor a komponens betölt automatikusan lekérdeződik a beszélgetések listája a **/api/messages/convesations** API-on keresztül és megjelenik időrendi sorrendben, valamint az olvasatlan üzenetek száma a beszélgetések mellett, ezeket az olvasatlan számokat összeadva jeleníti meg a szülő komponens a navbar-ban az olvasatlan üzenetek számát.

- **Beszélgetés megnyitása:** amikor a felhasználó kiválaszt egy beszélgetést meghívásra kerül a **/api/messages/conversations/:id** API, ami visszatéríti az összes adatát az adott beszélgetésnek. Valamint egyből utána meghívásra kerül a /**api/messages/conversations/:id/read** API, ami az SQL táblában 0-ra állítja az adott beszélgetéshez tartozó unread számot úgy, hogy közben a másik félé érintetlen marad.

- **Üzenet küldése:** amikor a felhasználó elküld egy üzenetet, amely meghívja a /**api/messages/conversations/:id/messages** API-t, ez t követően a backend-en beszúr egy új sort a **messages** táblába és a beszélgetéshez tartozó sort frissíti a **conversations** táblában, hogy az utolsó üzenet dátuma az aktuális legyen. Ez után beiktat egy Socket.io eseményt.

- **Üzentek fogadása valós időben:** komponens betöltésekor az oldal létrehoz egy **Socket.io** kapcsolatot, amelynek az autentikációja a jwt tokennel történik és az ebben található adatok alapján azonosítja a felhasználót a backend. Amikor új üzenet érkezik az üzenet előnézet, illetve az olvasatlan szám frissül a state-ek között, ezután ellenőrzi, hogy az új üzenet a **selectedConversationRef**-ben lévő beszélgetéssel egyezik-e, ha igen akkor azüzenet a többi alá megjelenik, ha nem akkor az olvasatlan szám növekszik.

- **Beszélgetés törlése:** meghívásra kerül a **/api/messages/conversations/:id** API, amely kitörli a táblából a beszélgetést, valamint a frontend is kitörli state-ből a beszélgetést.

Foglalási folyamat(booking-flow)

Leírás: a felhasználók egy 4 lépcsős folyamaton keresztül tudnák foglalni időpontot a szalonokhoz, ehhez szükséges a bejelentkezés. Végig kiséri a felhasználót 4 lépésen – szolgáltató kiválasztása, szolgáltatás kiválasztása, időpont kiválasztása, véglegesítés.

Érintett fájlok és adatbázis táblák:

- **Frontend komponensek:**
  - **Bookingsheet.jsx:** Fő komponens, több lépcsőben jeleníti meg a foglalási folyamatot, amely **step** statet használja a lépések követésesre és renderelésésre.

  - **Salonpage.jsx:** Bookingsheet szülőkomponense, ez a szalon fő oldala itt jelennek mega szalon részletes infói.

  - **ProviderCard.jsx:** szolgáltatóhoz tartozó kártya, amelyet megnyomva a kiválasztott szolgáltatóval való foglalásra dobja a felhasználót.

- **Adatbázis táblák:**
  - **Appointments:** Foglalások adatai
  - **Users:** felhasználók adatai
  - **Providers:** szolgáltatók adatai
  - **Services:** Szolgáltatások, amiket a szolgáltatók nyújtanak
  - **Salons:** szalonok adatai
  - **Ratings:** értékelések
  - **Provider_time_blocks:** szolgáltatók szünetei, illetve nem elérhető időszakok

API végpontok:

Method	Útvonal	Leírás
GET	/api/search/salon/:id	Teljes szalon információ
GET	/api/search/salon/:id/reviews	értékelések
GET	/api/user/profile	Betölti a felhasználó adatait
GET	/api/user/Provider/:providerid/availability	Mutatja az elrhető időponttokat
POST	/api/user/appointments	Létrehoz egy foglalást
POST	/api/waitlist	Várólistához csatlakozás

Működési logika:

- **Salonpage:** minden szalonnak van oldala, ami a **SalonPage.jsx-en** keresztül jelenik meg. Ez a komponenes betöltéskor meghívja a **/api/search/salon/:id** végpontot és betölti a szalon összes adatát valamit a hozzá tartozó szolgáltatók és szolgáltatások adatait. Ezeket az adatokat később prop-ként adja tovább a **Bookingsheet.jsx**-nek.
- **Booking flow:** a foglalási folyamat több lépésen kíséri át a felhasználót, az aktuális lépést egy state-ben tárolja a rendszer, nem történik API hívás mivel minden adat már memóriában van.
    - **Szolgáltató kiválasztása:** a szolgáltatók listája kerül megjelenítésre, valamint egy bármelyik szolgáltató gomb, ezekere kattintva lépünk tovább a következő lépésre.

    - **Szolgáltatás kiválasztása:** megjelenítésre kerülnek a kiválasztott szolgáltató szolgáltatásai árral és időtartammal együtt. Ha a felhasználó a bármelyik szolgáltató opciót választott úgy az összes, a szalonhoz kapcsolódó szolgáltatás megjelenik.

    - **Időpont kiválasztása:** egy mini naptár kerül renderelésre ahol csak az adott szalon által elérhetővé tett napok kerülnek megjelenítésre. Amikor a felhasználó kiválaszt egy dátumot, a **/api/user/provider/:providerid/availability?date=YYYY-MM-DD&ServiceDuration=XX** kerül meghívásra és megjeleníti az elérhető időpontokat.
        - **Backend logika(availabilityservice.**js):
            - lekérésre kerül a szalon nyitvatartási ideje
            - 15 perces intervallumokat generál a záró és nyitó idő között
            - Ezek után törlésre kerülnek a múltbeli, másik időpontot vagy szolgáltatói szünetet keresztező, valamint a nyitvatartási időn túl lógó szolgáltatásokat
            - Majd visszatéríti az eredményt

- **Véglegesítés:** megjelenítésre kerülnek a foglalás és a felhasználó adatai, valamint egy véglegesítés gomb.

    - **Backend logika (database transaction):**
        - **Bemenet ellenőrzése:** az időpontnak a jövőben kell lennie, valamint a szalon nyitvatartási napjain belül
        - **Szolgáltató ellenőrzése:** ellenőrzi, hogy szolgáltató többi időpontjával ne ütközzön.
        - **Felhasználó ellenőrzése:** ellenőrzi, hogy a felhasználó egyik másik időpontjával sem ütközik-e, valamint kiszámol egy utazási időt az előző időpont helyszínéhez képest és figyelmezteti a felhasználót az esetlegesen túl közel foglalt időpontokról.
        - **Időpont létrehozása:** időpont létrehozása után commit-ra kerül a transaction
- **Várólista:** ha nincs elérhető időpont az adott idő intervallumban a felhasználó tud egy várólistához csatlakozni. A felhasználó kiválaszthat egy intervallumot, ahol, ha egy szabad időpont jön létre, email értesítést kap arról.

- **Háttér folyamat:**
  AppointmentExpirationService.js 5 percenként lefut és „completed”-re írja át a foglalás státuszát ha az a múltban van.

Szolgáltatói (Provider) Modul

Szolgáltatási Belépési Pont (User Journey)

Mielőtt a szolgáltató (Provider) elkezdené a tényleges regisztrációs folyamatot, a felhasználói út (User Journey) a publikus **ProvLanding.jsx** és **Slideshow.jsx** komponenseken keresztül kezdődik. Ezek a felületek elsősorban marketing és vizuális reprezentációs célokat szolgálnak: röviden bemutatják az alkalmazás főbb funkcióit, valamint a választható szolgáltatói előfizetési csomagokat (alap, pro, premium). Mivel ezek az oldalak nem igényelnek hitelesítést és nem végeznek komplex adatbázis-műveleteket, a fejlesztés hangsúlya itt a reszponzív, vonzó megjelenítésen, illetve a felhasználó zökkenőmentes átirányításán van a tényleges Regisztráció/Onboarding folyamat felé.

Regisztráció és Bejelentkezés (Provider Onboarding)

Leírás:

A szolgáltatói (Provider) modul onboarding folyamata két fő ágon indulhat el: új szalon létrehozása vagy csatlakozás egy meglévőhöz. Az alkalmazás **nem FormData multipart feltöltést** használ a regisztráció során az alapvető adatoknál, hanem tiszta **JSON alapú** kommunikációt. A folyamat lépésekre bontva vezeti végig a felhasználót a szalon adatainak és a személyes fiók adatainak megadásán. A bejelentkezés modern, dupla tokenes (Access + Refresh token) stratégiára épül a biztonságos session kezelés érdekében.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **ProvRegister.jsx:** Három lépésből álló, előre definiált állapotokra épülő regisztrációs felület (CHOICE -> JOIN_SALON vagy CREATE_SALON -> USER_REGISTRATION). Belső state-ek tartják karban a kiválasztott adatokat, a komponens natív fetch hívásokkal kommunikál a backenddel.
  - **ProvLogin.jsx:** Az e-mail/jelszó bekéréséért felelős egyetlen űrlap. Sikeres hívás után beállítja az Access Tokent a localStorage-ben, és a useAuth hook hívásával globálisan is bejelentkezett státuszba teszi az alkalmazást.
  - **AddressInput.jsx:** Ez az összetett komponens köti össze a beviteli mezőt (ahol a hook-ból kapott javaslatok megjelennek) a térképes vizualizációval. Ha a felhasználó kiválaszt egy címet, a komponens elmenti annak koordinátáit.
  - **useAddressAutocomplete.js:** Újrafelhasználható egyedi (custom) hook az autocomplete logikához. Hívásokat késleltet (debounce mechanizmus, alapértelmezetten 400 ms, minimum 3 karakteres lekérdezés mellett), majd a saját backend GET /api/search/address-autocomplete proxy végpontján

keresztül kéri le a javaslatokat. Maga a Nominatim (OpenStreetMap) hívás szerveroldalon történik, így a kliens nem közvetlenül a külső szolgáltatást szólítja meg. A Provider címkeresés debounce-logikáját a useAddressAutocomplete.js hook belső időzítője kezeli.

- **BaseMap.jsx:** (A Térképes Infrastruktúra alapja) Az AddressInput nem maga építi fel a térképet, hanem ezt a közös "burkoló" (wrapper) komponenst használja. A rendszer két fő részre támaszkodik:
  - **React-Leaflet:** Ez felel a térkép interaktív "motorjáért" (húzás, nagyítás, kattintható jelölők elhelyezése). Ez köti össze a térképet a React rendszerével.
  - **MapTiler:** A Leaflet önmagában csak egy üres felület lenne. A vizuális utcatérképet (az apró képrészleteket, azaz "csempéket") a MapTiler külső szolgáltatás adja egy API kulcson keresztül.
  - **Technikai extra:** A BaseMap felel a teljesítmény javításáért is (megakadályozza a térkép felesleges újratöltődését gépelés közben), illetve itt vannak fixen beállítva a térképes jelölőikonok (markerek), megelőzve az eltörő kép-útvonalakból adódó csomagolási (bundler) hibákat.
- **frontend/src/modules/auth/auth.js** (és **App.jsx bekötések**): A kliens oldali token frissítések és a bejelentkezett állapot globális Context-ben való tartásáért felelősek.

- **Adatbázis táblák (MySQL):**
  - **providers:** A szolgáltatói fiókok táblája. Tartalmazza a salon_id-t, a role-t (lehetséges értékei: „manager”, „provider”), az isManager logikai mezőt, valamint a status-t (pl. „active”, „inactive”, „deleted”, „banned”), továbbá a belépési adatokat. Emellett tárolja a fiók létrehozásának időpontját, a profile_picture_url-t (kezdetben NULL értékkel), valamint az utolsó bejelentkezés időpontját (last_login), ami szintén NULL érték kezdetben.
  - **salons:** A szalonok paraméterei, amellyel a provider össze van kötve (salon_id külső kulccsal). Létrehozáskor kap egy generált 6 karakteres egyedi sharecode-ot a backendtől (crypto.randomBytes(3).toString('hex').toUpperCase(), így 0–9 és A–F karakterekből áll, pl. „A1F3C9”), illetve a Nominatim által címből átalakított latitude és longitude koordinátákat.
  - **RefTokens:** A bejelentkezéskor generált refresh tokenek tárolásához használt tábla. Az insertProviderRefreshToken művelet ide rögzíti a kibocsátott tokent, így a rendszer szerveroldalon is képes érvénytelenné nyilvánítani azt.
  - **system_logs:** a sikeres regisztrációkat (PROVIDER_SIGNUP, SALON_CREATED új szalon esetén, PROVIDER_SALON_JOIN csatlakozás esetén) és belépéseket (PROVIDER_LOGIN) a services/logService.js logEvent függvénye rögzíti ide.

API végpontok (backend/api/auth/ProvLoginApi.js):

METHOD	Útvonal	Leírás

POST	/auth/provider/validate-salon-code	Egy megadott 6 karakteres sharecode érvényességét ellenőrzi az adatbázisban mielőtt továbbenged a személyes regisztrációhoz.
POST	/auth/provider/register	Kiszolgálja mind a 'join', mind a 'create' ágat egy tranzakció keretében. Létrehozza a providers és (szükség esetén) a salons rekordot.
POST	/auth/provider/login	Provider hitelesítése, jelszó-hash összehasonlítása. Sikeres belépés esetén visszaadja a rövid életű JWT access tokent, és a válaszfejlécben egy HttpOnly cookie segít beállítani a refresh tokent.

Működési logika és Adatfolyam

- **Regisztráció (ProvRegister.jsx):**
  - A komponens előre definiált állapotokon (STEPS) megy keresztül egy React switch-case segítségével kicserélve a nézetet, elkerülve ezzel a teljes oldalújratöltést vagy route váltást a regisztráció alatt.

  - **Szalon létrehozás / Csatlakozás választás (CHOICE):**
    - A felhasználó gombra kattintása beállítja a registrationType state-et join vagy create értékre.
  - **Join (JOIN_SALON) Ág:**
    - Függvény: handleValidateSalonCode()
    - A kliens elküld egy egyszerű objektumot ({ code: salonCode }) a /validate-salon-code végpontra.
    - ***Backend válaszkódok:***
      - 200 OK: Ha a kód létező és a szalon státusza nem `closed`, visszaadja a { salonId, salonName } párost (ezt a frontend elmenti az állapotba, hogy kiírhassa, melyik szalonhoz fog csatlakozni, és meg is tartja a végleges küldéshez a salonId-t).
      - 404 Not Found: Ha nem létezik a kód (Megjelenik az error banner a frontenden: 'Nem található szalon ezzel a kóddal').
      - 400 Bad Request: Túl rövid kód.
  - **Create (CREATE_SALON) Ág:**
    - A felhasználó kitölti a szalon nevét, leírását, és az AddressInput-on keresztül bekéri a címet. A komponens helyben biztosítja, hogy kinyerje a latitude és longitude paramétereket az OpenStreetMap-ből.
    - Függvény: handleSalonDataSubmit() egyelőre csak kliens oldalon ellenőriz, és ha valid a mező, továbbléptet a 3. lépésre.

- **Személyes adatok és végleges beküldés (USER_REGISTRATION):**
  - Függvény: handleFinalRegister()
  - Ekkor áll össze a végleges JSON payload. Nem multipart/form-data jön létre!
  - A kérés formátuma csatlakozás esetén: { name, email, phone, password, registrationType: 'join', salonId }.
  - Létrehozás esetén: { name, email, phone, password, registrationType: 'create', salon: { companyName, address, description, salonType, latitude, longitude } }.
  - Backend oldali folyamat (ProvLoginApi.js): Megnyílik egy MySQL tranzakció (pool.getConnection().beginTransaction()). Létrehozás (create) esetén a szerver egy beépített biztonsági tartalék (fallback) logikát is alkalmaz: ha a frontend valamiért pontos koordináták (lat/long) nélkül küldené be az adatot, a locationService.placeToCoordinate() szerveroldali metódus másodlagosan megpróbálja geokódolni a címet. Új szalon létrehozásakor a providers táblában az új szolgáltató role = 'manager' és isManager = true értéket kap. Csatlakozás esetén role = 'provider' és isManager = false kerül mentésre. Fontos, hogy a provider JWT token role claimje ettől függetlenül minden szolgáltatónál provider; a manager-only műveleteket az adatbázisból ellenőrzött isManager mező és az isManagerMiddleware védi. A folyamat végén a szerver a services/emailService.js sendWelcomeEmail szervize aszinkron módon üdvözlő e-mailt küld a szolgáltatónak (ez egyúttal a sikeres regisztráció visszaigazolásaként is szolgál), majd bejegyzést generál a system_logs táblába.
  - ***Backend válaszkódok:***
    - 201 Created: Sikeres folyamat. A React router egy időzítést (timeout) követően átirányítja a felhasználót a bejelentkezési felületre.
    - 409 Conflict: Két esetben lép fel (a kliens pontosan tudja jeleníteni a hibaüzenetet): (1) Az email cím/telefonszám már létezik. (2) Új szalon esetén ugyanezzel a névvel ezen a címen már létezik rekord.
    - 400 Bad Request: Hiányzó mezők, jelszó 8 karakternél rövidebb backend oldali ellenőrzés alapján (bár a frontend is szűri).
    - 500 Internal Server error: DB tranzakciós vagy rollback hiba.

- **Bejelentkezés (ProvLogin.jsx):**
  - A hitelesítés egy külön oldalon és API híváson keresztül fut a regisztráció után.
  - **Űrlap elküldése:**
    - A handleLogin függvény JSON teste { email, password }. A fetch credentials: 'include' beállítással kerül elküldésre, ami elengedhetetlen feltétele annak, hogy a szerver által küldött HttpOnly cookie-kat fogadja a böngésző.
  - **Szerveroldali logika (ProvLoginApi.js /login):**

        - Megkeresi az adatbázisban a felhasználót.
        - Ellenőrzi a jelszó validitását bcrypt.compare segítségével.
        - Sikeres jelszó-egyezés után a backend frissíti a providers.last_login mezőt az aktuális időbélyeggel, majd generál egy JWT accessToken-t és egy JWT refreshToken-t az authUtils.js belső segédmodul használatával. Elmenti a refresh tokent a DB-be.
    - **Válaszkezelés:**
        - 200 OK: A válaszban (response.json()) jön az accessToken és a provider alapadatai. A frontend az accessToken-t a localStorage-be menti. A refreshToken ezzel szemben kizárólag HttpOnly cookie-ban érkezik vissza, így a kliensoldali JavaScript nem fér hozzá – ez csökkenti az XSS-támadások hatókörét a refresh tokenre nézve. A refresh token HttpOnly cookie-ban érkezik; production környezetben Secure attribútummal, SameSite=Lax beállítással. Sikeres authentikációt követően átirányít a /ProvDash oldalra.
        - 422 Unprocessable Entity: Sikertelen belépéskor, szándékosan egy egységes "Hibás email vagy jelszó" üzenettel tér vissza függetlenül attól, hogy a jelszó nem stimmelt vagy az email címet nem találta (felhasználó-enumerációs támadások megelőzése érdekében).
        - 403 Forbidden: A hitelesítés akkor is megszakad, ha a Provider státusza banned vagy deleted. A válasz tartalmaz egy banned: true flaget és egy reason mezőt, amelynek értéke a fiókállapotot (kitiltás vagy GDPR-törlés) jelzi, így a frontend külön üzenetet jeleníthet meg a kitiltott és a GDPR-törölt fiókok esetén.
        - 400 Bad Request: Valamely beviteli mező üresen maradt, bár ezt a kliensoldali validateInputs függvény már előzetesen ellenőrzi.

Irányítópult (Dashboard) – Fő nézet és Navigáció

Leírás:

A ProvDash a szolgáltatói alkalmazás központi keret-komponense (layout-konténere) és fő vezérlőpultja. Sikeres bejelentkezés után a szolgáltató ide érkezik. Ez a komponens felelős a globális navigációért, a felső fejléc (header) megjelenítéséért a bejelentkezett felhasználó profiljával és legördülő menüjével, továbbá ez a komponens tölti be dinamikusan a kiválasztott menüpontnak megfelelő tartalmat (például a naptárat, ügyféllistát, vagy a kezdő áttekintő nézetet). Az alapértelmezett kezdőképernyő az Áttekintés (OverviewSection), amely statisztikai adatokat mutat. Itt található a profil szerkesztésére és a jelszó módosítására szolgáló modal ablakok megnyitási logikája is.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**

- **ProvDash.jsx:** A fő keretrendszer. Olyan állapotokat (state) kezel, mint az éppen aktív tab (activeTab), a lenyíló profilmenü állapota, és ő felelős a bejelentkezett szolgáltató alapadatainak (profilkép, név, role) kéréséért a betöltődéskor. A komponens emellett kezeli a globális messagesUnread számlálót (a MessagesSection onUnreadChange propján keresztül), valamint a pendingConversation átadást: ha a felhasználó a Vendégkör vagy a Naptár modulból nyit chatet, a Dashboard a messages tabra kapcsol és átadja a már megnyitott vagy létrehozott beszélgetést.
- **provdashcomponents/NavButton.jsx:** A fő térben elhelyezkedő navigációs gombok ismétlődő UI dizájnját valósítja meg.
- **provdashcomponents/UserDropdown.jsx:** Profil szerkesztése, Beállítások és Kijelentkezés elemeket jelenít meg; a jelszómódosítás a ProfileModalból érhető el.
- **provdashcomponents/OverviewSection.jsx:** Az alapértelmezett, kezdeti statisztikai képernyő a mai foglalások számával, az aktuális heti bevétellel és a közelgő (elkövetkezendő) időpontok listájával. Ebben a komponensben ún. Skeleton-loaderek is találhatók: A betöltési idő alatt a Megosztott háttér-szolgáltatások szekcióban ismertetett Skeleton-loader komponenscsalád használatos.
- Ebből a szülő komponensből nyílnak meg, de logikailag a következő témakörhöz fognak tartozni a profil alapadataival és profilképpel foglalkozó ProfileModal és jelszóváltoztatást kezelő PasswordModal is.

- **Adatbázis Táblák (MySQL):**
  - A Dashboard a specifikus aloldalak (tabok) rendereléséig minimális adatbázis-műveletet indít: ezen a legfelső állapotszinten csupán a bejelentkezett szolgáltató alapadatait és jogosultsági jelölőit (például az isManager logikai flageket) kéri le, amelyek elengedhetetlenek a globális navigációs menüpontok hozzáférhetőségének szabályozásához.
  - **providers:** az aktuális szolgáltató adatainak (pl. isManager), új jelszavának eltárolása miatt.
  - A statisztikához (az OverviewSection-ben) több tábla JOIN-jából, összetett lekérdezéssel jönnek az adatok appointments, services és users táblákból.

API végpontok (backend/api/salon/providerProfileRoutes.js és backend/api/calendar/dashboardRoutes.js):

METHOD	Útvonal	Leírás
GET	/api/salon/me	**(providerProfileRoutes.js)** A ProvDash.jsx mountolásakor hívja meg. Lekéri az aktuálisan bejelentkezett provider alapadatokat. Elmenti a providerProfile state-be. (Profilkép URL, név, jogkör alapján való tiltások, pl. staff tab-hoz a isManager státusz).

GET	/api/provider/calendar/statistics	**(dashboardRoutes.js)** Az OverviewSection.jsx hívja le. Fontos statisztikai mutatókat (Mai foglalások száma, heti tervezett bevétel, új regisztrált ügyfelek száma az aktuális naptári hónap kezdete óta (a users.created_at mező alapján, a szolgáltatóhoz tartozó foglalásokon keresztül) és egy tömböt ad vissza a következő időpontok (upcoming) listájával.
PUT	/api/salon/me/password	**(providerProfileRoutes.js)** Lecseréli a bejelentkezett felhasználó jelszavát a jelszómódosító modal űrlapján keresztül ellenőrizve a jelenlegi jelszó helyességét az adatbázisban tárolt password_hash-hez képest.

Működési logika és Adatfolyam

- **Fő keretrendszer (ProvDash.jsx):**
  - **Navigáció (Tab-rendszer):** A komponens egy activeTab nevű state változót használ (alapértéke: 'overview'). A bal oldali menüsorban (vagy mobilon esetleg alul/fent) lévő gombok (NavButton) rákattintása megváltoztatja ezt az értéket. A fő tartalmi zónában (renderContent függvény) egy switch-case szerkezet ennek az értéknek megfelelően választja ki és rendereli újra a tartalmi zónába kerülő komponenst (pl. <CalendarSection />), React Router-váltás vagy oldalbetöltés nélkül.
  - **Manager jogosultság ellenőrzése (isManager check):** Ha a bejelentkezett szolgáltató nem rendelkezik manager jogosultsággal (providerProfile.isManager === false), a rendszer kliensoldalon letiltja a StaffManagement elérését. A handleStaffTabClick meghívásakor a komponens a providerProfile.isManager értékét vizsgálja, és jogosultság hiányában error típusú Toast értesítéssel jelzi a korlátozást.
  - **Globális események (Click Outside):** Van egy useEffect, ami a profil legördülő menüjére figyel. Ha meg van nyitva a lenyíló menü, és a felhasználó bárhova máshova kattint az oldalon, a menü automatikusan visszazáródik egy mousedown eseményfigyelővel. Ebből a lenyíló menüből intézhető továbbá a kijelentkezés POST `/auth/logout` hívással törli a szerveroldali refresh tokent és cookie-t, majd a kliens eltávolítja az access tokent.
- **Áttekintő szekció (OverviewSection.jsx):**
  - Amikor betölt, rögtön meghívja a /api/provider/calendar/statistics API részt. A hívás a közös autentikált hálózati wrappert (az authApi-t) használja, amivel hitelesített marad.
  - Ameddig a loading state igaz állapotú, a végleges adatok érkezéséig a Megosztott háttér-szolgáltatások szekcióban ismertetett Skeleton-loader komponensek jelennek meg helyőrzőként a kártyák és listák helyén.

- A visszaérkező adatot:
  - Feldolgozza az Intl.NumberFormat-al magyar forint (hu-HU) ábrázolásba. Ezt a weeklyRevenue mezőnél használja fel.
  - Az upcomingAppointments backend oldalon csak jövőbeni, scheduled státuszú foglalásokat tartalmaz. A frontend status-color helper általánosabb, de az Overview jelenlegi adathalmaza scheduled upcomingokra szűkül. A getStatusColor metódus automatikusan legenerálja az adott gombok, feliratok (badge-k) zöld, kék vagy piros színárnyalatait az eredménytől függően. Továbbá a felhasználó (vendég) nevének kezdőbetűiből (pl.: Kovács János -> KJ) generál egy profil-monogramot vizuális elemként.

Szalon Profil és Arculat Menedzsment

Leírás:

A szalon menedzsment modul biztosítja, hogy a megfelelő jogosultsággal (manager) rendelkező szolgáltatók testreszabják szalonjuk publikus megjelenését. A modul lehetőséget nyújt a cég alapadatainak (név, cím, nyitvatartás, elérhetőségek) módosítására és aktív/inaktív állapotának kezelésére. Ezen felül vizuális arculatkezelést (szalon logó, promóciós banner, márkaszín) is támogat FormData alapú képfeltöltéssel, ami azonnal látható a vendégek (Customer) keresőfelületén.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **SalonManagement.jsx:** Ez a fő konténer elem felel a szalon adatainak betöltéséért, a szerkesztési mód (editMode) kezeléséért és az adatok beküldéséért. Integrált logikája azonnal figyeli és előnézetben (preview) jeleníti meg a feltöltött képeket, miközben képarány (aspect ratio) validációt is végez a banner esetében.
  - **AddressInput.jsx:** Segédkomponens, amely szabványosítja a szalon fizikai címének bekérését, így megelőzve az elgépeléseket a földrajzi helymeghatározáshoz.
- **Adatbázis táblák (MySQL):**
  - **salons:** A szalonok fő táblája. Tárolja az alapadatokat (**name, address, phone, email**), a nyitvatartást (**opening_hours, closing_hours, open_days** JSON formában) és a vizuális tulajdonságokat (**banner_color, logo_url, banner_image_url**). A státuszokat is itt rögzítjük az status (**open, closed, renovation**) mezőben.
    - **Megjegyzés:** a regisztráció során beérkező salonType érték a type oszlopba kerül; a PUT /api/salon/update végpont is ezen a type mezőn keresztül kezeli a szalon kategóriáját.

- **providers:** Ez is része a logikának közvetve. A jogosultságokat ellenőrizzük (hogy a belépett provider isManager mezője true-e), ami vizuálisan és backend oldalon is limitálja a funkcióhoz való hozzáférést.

API végpontok (backend/api/salon/settingsRoutes.js):

METHOD	Útvonal	Leírás
GET	/api/salon/my-salon	Visszaadja a belépett szolgáltató szalonjának minden adatát, valamint informál az aktuális felhasználó jogosultsági (isManager) szintjéről a UI rendering eldöntéséhez.
PUT	/api/salon/update	fogadja és frissíti a szalon szöveges és numerikus alapadatait (pl. cím, leírás, telefonszám), szigorúan manager szintű jogosultság ellenőrzése után. A name és address mező megadása kötelező; a phone, email, type, description, latitude, longitude, opening_hours, closing_hours opcionális, az open_days tömböt a backend JSON.stringify-olja a tárolás előtt.
PUT	/api/salon/status	Egyetlen új státuszt vár body-ban (status), melyet megváltoztatva a szalon akár teljesen inaktívvá tehető a foglalási rendszerben a vendégek elől.
POST	/api/salon/branding	Képfeltöltést (logó, banner jpg/png/webp) és a márkaszínt (banner_color) fogad FormData multipart/form-data küldéssel. Mentés után visszatér a frissített salon objektummal, amely tartalmazza az új logo_url, banner_image_url és banner_color értékeket. A banner_color mezőre a backend reguláris kifejezést alkalmaz (/^#[0-9A-Fa-f]{6}$/); érvénytelen szín 400-as hibát eredményez.
POST	/api/salon/branding/remove-banner	Eltávolítja a szalon jelenleg mentett promóciós bannerét mind az adatbázisból, mind a szerver tárhelyéről, visszaállítva az alapértelmezett megjelenést.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés:**
  - A komponens betöltésekor az alkalmazás a GET /api/salon/my-salon végpontról lekérdezi az alapadatokat.
  - A válaszból nemcsak a cégadatok épülnek be a lokális state-be (salon), hanem a rendszer megvizsgálja azt is, hogy a belépett felhasználó menedzser-e

(isManager). Ha nem, a felület 'Read Only' módban kerül megjelenítésre, kizárva a jogosulatlan módosításokat.

- **Alapadatok szerkesztése:**
  - A 'Szerkesztés' bekapcsolásával az űrlap mezői egy különálló formData lokális React state-re csatlakoznak. Ez biztosítja, hogy a gépelés közbeni módosítások ne írják felül azonnal az aktív salon state-et, így a felhasználó következmények nélkül megszakíthatja a folyamatot (Mégsem).
  - Mentéskor az alkalmazás a teljes formData objektumot – a frissített értékekkel együtt – továbbítja a PUT /api/salon/update végpontra. Sikeres hívás esetén a memóriában lévő fő állapot is frissül, a felület pedig automatikusan kilép a szerkesztő módból.
- **Arculati menedzsment (Képfeltöltés):**
  - Mikor a felhasználó kiválaszt egy logót vagy banner képet, a natív <input type="file" /> aktiválja az eseménykezelőt (pl. handleBannerSelect()).
  - Új logó vagy banner kiválasztásakor azonnali kliensoldali validáció fut le: a rendszer ellenőrzi a fájl kiterjesztését és méretét (.jpg, .png, .webp és legfeljebb 5MB) (validateImageFile()).
  - Sikeres ellenőrzés után a böngésző egy ideiglenes memóriacímet (URL.createObjectURL) generál az előnézethez, és a háttérben megvizsgálja a kép méretarányát (aspect ratio). Ha a banner formátuma túlságosan eltér a kívánttól, a UI figyelmeztető üzenetet jelenít meg a lehetséges torzulásról.
  - Mentéskor a rendszer egy FormData objektumot állít össze, amely képes a bináris képfájlok és a szöveges adatok (márkaszín) egyidejű és biztonságos beágyazására. Ezt az adatcsomagot utána a POST /api/salon/branding végpontra küldi, ahol a szerveroldali middleware gondoskodik a képek fizikai mentéséről. Szerveroldalon a processSalonLogo/processSalonBanner Sharp-alapú feldolgozó újraméretezi és a megengedett MIME típusokra (image/jpeg, image/png, image/webp) szűri a fájlt, majd a deleteOldSalonImage törli a salons.logo_url/banner_image_url mezőben szereplő korábbi képet a fájlrendszerből.
- **Láthatóság menedzsment (Státusz váltás):**
  - A szalon állapotának megváltoztatása (pl. „Nyitva”, „Zárva”, „Renoválás”) három, egymás mellett elhelyezett gombbal történik. Egyszerre csak egy állapot lehet kiválasztva. A kiválasztott gombra kattintva azonnal egy PUT /api/salon/status kérés indul, így a változtatás azonnal érvénybe lép, függetlenül a fő űrlaptól.

Alkalmazottak és Csapat Kezelése (Staff Management)

Leírás:

Ez a modul teszi lehetővé a szalon menedzserei számára a szalonhoz tartozó szolgáltatók (alkalmazottak) áttekintését és adminisztrációját. A felületen listázhatók a csapattagok, módosítható a státuszuk (aktív/inaktív), menedzseri jogkör (isManager) adható vagy

vonható meg tőlük, eltávolíthatók a csapatból soft delete művelettel, vagyis a provider státusza deleted értékre változik. A nézet egy integrált naptárat is biztosít az adott munkatárs egyéni beosztásának és foglalásainak áttekintéséhez.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **StaffManagement.jsx:** A csapatkezelő fő konténer komponense. Lekéri a szalonhoz tartozó szolgáltatókat a /api/salon/providers végpontról, kezeli a kiválasztott munkatársat (selectedStaff), az aszinkron műveletek állapotát (actionLoading), valamint mobil nézetben a lista és részletes panel közti váltást (showDetailOnMobile). Innen indulnak a státusz-, menedzseri jog- és eltávolítási műveletek.
  - **StaffCard.jsx:** A bal oldali munkatárslista egyes elemeit jeleníti meg. Kezeli az avatárt vagy monogramot, az aktív/inaktív/törölt státusz jelzését, a menedzseri jelölést, valamint az adott munkatárshoz tartozó foglalásszám megjelenítését.
  - **StaffCalendarView.jsx:** A kiválasztott munkatárs részletes naptárnézete. A /api/staff/calendar/:staffId végpontról tölti be az adott nap foglalásait és time blockjait, a /api/staff/services/:staffId végpontról pedig a munkatárs aktív szolgáltatásait. Manager jogosultsággal innen új foglalás is létrehozható az adott munkatárshoz, illetve meglévő foglalás törölhető.
  - **CalendarSection/CreateAppointmentModal.jsx:** Új foglalás létrehozására használt újrahasznosított modal komponens. A StaffCalendarView is ezt használja, de staff kontextusban a mentés nem a provider saját calendar endpointjára, hanem a /api/staff/appointment végpontra fut.
  - **CalendarSection/AppointmentDetailModal.jsx:** Foglalás részleteinek megjelenítésére és törlés indítására használt újrahasznosított modal komponens.
- **Backend route fájlok:**
  - **backend/api/salon/staffRoutes.js:** A szalon csapattagjainak listázását, módosítását és eltávolítását kezeli a /api/salon/providers és /api/salon/provider/:providerId útvonalakon.
  - **backend/api/staffApi.js:** A kiválasztott munkatárs naptárnézetéhez és manager általi foglaláskezeléshez tartozó külön staff API. Minden útvonala AuthMiddleware + isManagerMiddleware védelem alatt fut.
  - **backend/middleware/providerMiddleware.js:** Az isManagerMiddleware ellenőrzi, hogy a kérő menedzser-e, és beállítja a request.salonId értéket, amely alapján a staff API megakadályozza más szalon munkatársainak kezelését.
- **Adatbázis táblák (MySQL):**
  - **providers:** A csapattagok elsődleges táblája. Itt módosul a status, az isManager, valamint a role mező. A törlés soft delete: a provider státusza deleted lesz.

- **appointments:** A StaffCalendarView naptárnézete innen olvassa a kiválasztott munkatárs foglalásait, és ide kerülnek a manager által staff nevében létrehozott új foglalások.
- **services:** A staff naptárban új foglalás létrehozásakor ebből a táblából jönnek az adott munkatárs aktív szolgáltatásai.
- **provider_time_blocks:** A staff naptár megjeleníti az adott munkatárs szüneteit és kieső időszakait is.
- **users:** Staff appointment létrehozásakor regisztrált ügyfél esetén a foglalás meglévő Bookly userhez kapcsolódik e-mail alapján. Ha az ügyfél nem rendelkezik regisztrált fiókkal, a helyes folyamat szerint vendégfoglalásként (is_guest: true, név + e-mail + telefon megadásával) kell rögzíteni.
- **system_logs:** A /api/salon/provider/:providerId módosítások és törlések naplózására használatos (PROVIDER_TEAM_UPDATED, PROVIDER_TEAM_REMOVED).

API végpontok (backend/api/salon/staffRoutes.js és backend/api/staffApi.js):

METHOD	Útvonal	Leírás
GET	/api/salon/providers	Lekéri a belépett provider szalonjához tartozó összes szolgáltatót. Nem manager-only endpoint, de hitelesítés szükséges hozzá.
PUT	/api/salon/provider/:providerId	Egy adott csapattag adatait módosítja: név, telefon, leírás, státusz vagy menedzseri jogosultság. Manager-only végpont. Ellenőrzi, hogy a célzott provider ugyanahhoz a szalonhoz tartozik-e, és védi az utolsó menedzser önlefokozását.
DELETE	/api/salon/provider/:providerId	Soft delete művelettel eltávolítja a szolgáltatót a csapatból, vagyis status = 'deleted' értékre állítja. Manager-only végpont, és védi az utolsó menedzser öneltávolítását.
GET	/api/staff/services/:staffId	Lekéri a kiválasztott munkatárs aktív szolgáltatásait. A StaffCalendarView új foglalás modalja ezt használja a szolgáltatásválasztó feltöltésére. Manager-only, és ellenőrzi, hogy a staff tag a manager saját szalonjához tartozik-e.
GET	/api/staff/calendar/:staffId?date=YYYY-MM-DD	Lekéri a kiválasztott munkatárs adott napi foglalásait, time blockjait, valamint a szalon nyitási és zárási

		óráját. Manager-only, szalonhoz tartozás ellenőrzéssel.
POST	/api/staff/appointment	Managerként új foglalást hoz létre egy kiválasztott munkatárshoz. Ellenőrzi a staff szalonhoz tartozását, a szolgáltatás tulajdonjogát, a nyitvatartást, a foglalási ütközéseket és a time block ütközéseket.
PUT	/api/staff/appointment/:appointmentId	Staffhoz tartozó foglalás státuszát vagy megjegyzését módosítja. Manager-only, és ellenőrzi, hogy a foglalás a manager szalonjához tartozik-e.
DELETE	/api/staff/appointment/:appointmentId	Staffhoz tartozó foglalást töröl fizikailag az adatbázisból, ha az a manager szalonjához tartozik, és még nincs canceled státuszban.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés:**
  - Amikor a StaffManagement komponens mountolódik, a useEffect meghívja a fetchStaff függvényt, ami egy GET kérést indít a /api/salon/providers végpontra. A backend az AuthMiddleware-ből kiolvassa a lekérő azonosítóját (request.user.userId), kikeresi a hozzá tartozó salon_id-t, majd visszaküldi annak a szalonnak az összes csapattagját. A frontend ezt az adatot a staff state-ben tárolja és listázza.
- **Szerkesztési logika:**
  - A jobb oldali panelen lévő Művelet gombok (menedzseri jog adása/visszavonása, deaktiválás, eltávolítás) lenyomásakor a megfelelő eseménykezelők (pl. handleToggleManager, handleToggleStatus) hívódnak meg. Ezek elküldik az inverz állapotot (pl. ha isManager true volt, false-t küld) a PUT /api/salon/provider/:id végpontra.
- **API kommunikáció és biztonság:**
  - A módosító végpontokon (PUT, DELETE) a backend kritikus védelmi és logikai validációkat végez:
    - a végpont az **isManagerMiddleware** által védett.
    - Kereszt-tulajdonosi védelem: A backend ellenőrzi (targetProvider.salon_id !== request.salonId), hogy a menedzser biztosan a saját szalonjának dolgozóját módosítja-e.
    - Self-demotion elleni védelem: Ha egy menedzser saját magától próbálja a menedzseri jogát visszavonni vagy önmagát eltávolítani, a szerver lekéri a szalonhoz tartozó menedzserek számát (getManagerCount). Ha ő az utolsó és egyetlen menedzser, a backend 400 Bad Request hibával elutasítja a módosítást, megakadályozva, hogy a szalon irányítás nélkül maradjon.

- **UI viselkedés:**
  - Az aszinkron funkciók (adatküldés) idejére az adott gomb beállítja az actionLoading state-et (tartalmazva az ID-t és az akciót, pl. manager-12). Ez ideiglenesen letiltja (disabled) a gombot, és egy betöltést jelző állapotba vált (további kattintások megakadályozása). Válasz beérkezésekor a showToast értesíti a felhasználót, végül a komponens automatikusan újra is hívja a fetchStaff függvényt, hogy frissítse a globális listát és a UI azonnal tükrözze a backend állapotát. A UI reszponzív: asztali nézetben mind a lista, mind a részlet/naptár nézet látszik, mobil nézetben viszont az adott személy kiválasztásakor (showDetailOnMobile = true) a lista eltűnik, és csak a részletes panel veszi át a képernyőt.
- **Speciális logika (AI-naptár csatolása):**
  - Ha ki van választva egy adott selectedStaff, a felület beágyazza a StaffCalendarView komponenst. Ekkor a munkafolyamat átadja az azonosítót, így ez utóbbi komponens saját belső useEffect-jein keresztül már csak specifikusan ehhez a "célzott" al-munkatárshoz tartozó foglalásokat és szolgáltatásokat fogja bekérni az API-tól, így a menedzser pontosan láthatja, mikor és min dolgozik az adott csapattag.

Szolgáltatások és Árazás (Services CRUD) Kezelése

Leírás:

Ez a modul biztosítja a szolgáltatók számára a saját szolgáltatásaik és árazásaik teljeskörű adminisztrációját (CRUD: Create, Read, Update, Delete). A felületen láthatók a meglévő szolgáltatások adatai (név, ár, időtartam, státusz), újak hozhatók létre, a meglévők szerkeszthetők, vagy inaktiválhatók/törölhetők. Kiemelt funkciója, hogy az egyes szolgáltatásokhoz egyedi referencia képeket lehet feltölteni (FormData segítségével), ami jelentősen javítja a foglalási felületen a vendégek felhasználói élményét.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **ServicesSection.jsx:** A fő nézet, amely betölti és kártyás/hálós (grid) elrendezésben megjeleníti a szolgáltatásokat. Felelős a lista globális állapotának (services state) és a betöltési státuszok (loading) kezeléséért.
  - **ServiceFormModal.jsx:** Egy többcélú modal funkció, amely létrehozásra és szerkesztésre is szolgál (a kapott editingService prop alapján dönt a funkciójáról). Belső állapotban (formData) tartja számon a beírt adatokat, és külön logikával kezeli a referenciaképek listáját és azonnali feltöltési folyamatát (images state).
  - **DeleteServiceModal.jsx:** Egy egyszerű megerősítő modal, amely a véletlen törlés megakadályozására szolgál.
- **Adatbázis táblák (MySQL):**

- **services:** Itt tárolódnak maguk a szolgáltatások. Rögzíti, hogy melyik szolgáltatóhoz (provider_id) tartoznak, mi az aktuális állapotuk (status: 'available', 'unavailable'), az árat (price), a perc alapú időtartamot (duration_minutes), valamint a szöveges description mezőt.
- **service_images:** Kapcsolótábla a szolgáltatásokhoz, ahol a képek szerveroldali relatív elérési útvonalai (image_url) tárolódnak egyedi azonosítóval. A sort_order rögzíti a feltöltés sorrendjét; a UI nem támogat manuális átrendezést, a kliens ezen érték szerint listáz.

API végpontok (backend/api/calendar/serviceRoutes.js):

METHOD	Útvonal	Leírás
GET	/api/provider/calendar/services	Lekéri az aktuálisan bejelentkezett szolgáltató összes saját szolgáltatását.
POST	/api/provider/calendar/services	Új szolgáltatást hoz létre JSON payload alapján a kapott alapadatokkal.
PUT	/api/provider/calendar/services/:id	Egy meglévő szolgáltatás text/numerikus adatait frissíti az id alapján.
DELETE	/api/provider/calendar/services/:id	Eltávolítja az adott szolgáltatást, ha nincs hozzá tartozó aktív (scheduled) foglalás. Aktív foglalás esetén a végpont 400-as hibával válaszol, és a kliens nem tudja végrehajtani a törlést.
GET	/api/provider/calendar/services/:id/images	Egy adott szolgáltatáshoz tartozó referencia képek listájának lekérése.
POST	/api/provider/calendar/services/:id/images	Referenciaképek aszinkron feltöltésére szolgál FormData formátumban (multipart/form-data). A backend szolgáltatásonként legfeljebb 5 képet enged feltölteni; a hatodik feltöltés 400-as hibával áll meg. A képeket a Sharp-alapú processServiceImage middleware méretezi és menti a fájlrendszerre.
DELETE	/api/provider/calendar/services/:serviceId/images/:imageId	Egy konkrét feltöltött referenciakép eltávolítása az adott szolgáltatásról és a fizikai tárhelyről.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés:**
  - A ServicesSection betöltődésekor (mount) a useEffect meghívja a fetchServices() függvényt. A frontend a védett authApi.get() metódussal hívja a végpontot (tehát az Authorization headerben automatikusan utazik az Access Token). A lekérés ideje alatt a UI felületén egy Skeleton-loader jelzi a betöltési folyamatot. A kapott adatokat a sikeres válaszból a setServices rakja be a komponens state-jébe.
- **Szerkesztési logika:**
  - Ha a felhasználó az "+ Új Szolgáltatás" gombra vagy egy létező kártya specifikus módosítás gombjára kattint, meghívódik a handleOpenModal(service) függvény.
  - Ez a függvény új létrehozása esetén kiüríti a formData propot, míg létező entitásnál azonnal feltölti a formData-t (név, ár, státusz, stb.) a kiválasztott szolgáltatás paramétereivel, hogy a módosítás a ServiceFormModal.jsx-ben azonnal látható legyen.
- **API kommunikáció:**
  - Mentéskor a handleSave egy rövid kliensoldali validációt futtat (név nem lehet üres, ár nagyobb nullánál, időtartam legalább 5 perc). A backend ugyanezeket az értékeket szigorúan ellenőrzi: az időtartamnak 5 és 480 perc közé kell esnie, az árnak nagyobbnak kell lennie nullánál, ellenkező esetben 400-as hibával válaszol. Ha rendben van, a folyamat egy elágazáshoz (if-else) érkezik: ha létezik az editingService.id, akkor PUT (módosítás), ha nincs, akkor POST (létrehozás) hívás indul JSON body-val (ahogyan a Salon settingsnél is láttuk a JSON alapú szöveges adatoknál).
  - Művelet végeztével az alkalmazás azonnal újrahívja a fetchServices() funkciót, szinkronizálva az UI teljes lista nézetét a backend-del.
- **Speciális logika (Fájlfeltöltés - FormData):**
  - A szolgáltatás képek hozzáadása csak azután lehetséges, miután a szolgáltatás már sikeresen létrejött (mert szükség van az :id-ra). Új szolgáltatás felvitele után ezért a modal nem záródik be rögtön, hanem megkapja az új ID-t backendtől, átváltva "szerkesztő" nézetre.
  - Képfeltöltéskor (handleFileSelect esemény) a rendszer kikerüli a JSON struktúrát, és egy natív JavaScript FormData objektumba append-eli a kiválasztott fájlt (serviceImage néven). Ezt az egyedi stream-et az authApi.upload hívás tölti fel a szerverre.
  - Ezalatt a UI-on az aszinkronságot a uploadingSlots specifikus state kezeli. Beállít egy egyedi időbélyeg-kulcsot (Date.now()), és arra az időre az adott kép-helyőrzőben (slot) egy forgó betöltő ikon (spinner) fut. Sikeres visszatéréskor a state kibővül az új kép adataival, és a bélyegkép renderelődik.
- **UI viselkedés:**
  - Erős figyelmet kapott a prevenció (hibamegelőzés). Fájltörléskor és komplett szolgáltatás törlésekor a gombok letiltódnak (disabled állapot egy deleting vagy

saving state által), megakadályozva az API végpontok indokolatlan, többszörös hívását. Sikeres vagy elvetett művelet után a modálok automatikusan kitisztítják a lokális állapotukat (formData, képelőnézetek, betöltési flagek), így nem maradnak benn elavult referenciák a következő nyitásra.

Naptár és Elérhetőség (Calendar & Availability) Kezelése

Leírás:

Ez a komplex részleg felel a szolgáltatók beosztásáért és az időpontok kezeléséért. Két szorosan összekapcsolódó területből áll: egy nagynézetes kalendáriumból (ahol a meglévő foglalások listázódnak és új manuális időpontok vihetők fel a vendégek számára), valamint egy Elérhetőség / Szünet (Time Blocks) menedzserből, ami blokkolja a vendégek foglalófelületét az ebédszünetek, szabadságok és egyéb ismétlődő vagy egyszeri kieső idők esetén.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **CalendarSection.jsx:** A fő vizuális kalendárium renderelő komponense. A nézet hónapokat kezel, lekéri az adott időszak (start-end date) összes már rögzített foglalását, és külön vizuális indikátorral jelöli a teljesen betelt napokat (fully-booked-days).
  - **CreateAppointmentModal.jsx:** Egy interaktív modal, amely a szolgáltató által manuálisan rögzített vendégek (például helyszínen vagy telefonon bejelentkezők) új időpontjainak felvitelére szolgál.
  - **AppointmentDetailModal.jsx:** Egy létező foglalásra kattintva megnyíló részletező nézet. Ez a komponens a foglalás részleteit jeleníti meg, chatet nyithat regisztrált usernél, és `scheduled` foglalás törlését indíthatja. A státusz PATCH endpoint backend oldalon létezik, de jelenleg nincs Provider UI-bekötése.
  - **AvailabilityManagement.jsx:** Elkülönített nézet a szünetek listázására. Háromféle szűrést (all, recurring, one-time) tesz lehetővé, és mutatja, hogy mely szünetek ismétlődnek naponta vagy adott napokon (pl. heti szinten hétfő-szerda-péntek).
  - **TimeBlockModal.jsx:** A szünetek létrehozásánál/szerkesztésénél megjelenő modal. Kezeli a dedikált "Multiday" (többnapos szabadság) és a komplex "Recurring" (ismétlődő) logikát egy speciális űrlapon.
  - **timeBlocksService.js (Frontend):** A **src/services** mappában elhelyezkedő hálózati réteg, amely a React számára aszinkron adatlekérést biztosít a szünetek beküldéséhez.
  - **backend/services/availabilityService.js (Backend):** A Node.js szerveren futó üzleti logika, amely a szabad foglalási idősávok és a teljesen betelt napok számításáért felel. A meglévő appointments és provider_time_blocks adatok alapján kizárja a már foglalt vagy kieső időszakokat, és erre épül a fully-booked-days végpont működése.

- **backend/api/calendarApi.js:** a /api/provider/calendar/* prefix routereinek (appointment-, customer-, service-, dashboardRoutes) közös belépési pontja és middleware-láncának (AuthMiddleware → requireRole(['provider']) → verifyProvider) bekötése.

- **Adatbázis táblák (MySQL):**
  - **appointments:** Központi táblázat minden foglalásnak. Tartalmazza a provider_id, service_id, user_id (regisztrált customernél) mezőket, az appointment_start és appointment_end időbélyegeket, az árat (price), a Provider által rögzíthető comment mezőt, valamint a guest-foglalások adatait (guest_name, guest_email, guest_phone).
    A status ENUM értékei: scheduled, completed, canceled, no_show, deleted. A deleted érték a Provider modul jelenlegi végpontjain keresztül nem áll elő – a foglalások törlése a DELETE /api/provider/calendar/appointments/:id végponton fizikailag történik. A végpont 400-as hibát ad, ha a foglalás státusza már canceled. A softDeleteAppointment függvény (státusz deleted-re állítása + deleted_at/deleted_by rögzítése) az admin-oldali törlésekhez van fenntartva.
  - **provider_time_blocks:** Kifejezetten a rugalmas szabadidők táblája. Ha egyszeri szünetről van szó, sima metszet. Ha ismétlődő, a is_recurring logikai (boolean), a recurrence_pattern (daily, weekly) és a recurrence_days tartja nyilván az ismétlődési struktúrát JSON formátumban rögzítve a kiválasztott napokat.
    - **Megjegyzés:** a recurrence_days JSON formában tárolódik, de a MySQL-driver visszaolvasáskor egyes esetekben string formában adhatja vissza – a frontend ezért minden olvasásnál ellenőrzi, és szükség esetén JSON.parse-olja az értéket.
  - **waitlist:** Egy API logikailag szorosan ide kötött (rejtett) adatbázistábla, mely bizonyos betelt időpontokra feliratkozott vendégeket tartja számon várakozó státuszban.

API végpontok (backend/api/calendar/dashboardRoutes.js és backend/api/calendar/appointmentRoutes.js és backend/api/timeBlocksApi.js):

A naptár- és időblokk-végpontok minden útvonala védett. A middleware-lánc az AuthMiddleware → requireRole(['provider']) → verifyProvider sorrendet követi (a verifyProvider viselkedését lásd a Megosztott háttér-szolgáltatások szekcióban). Az isManagerMiddleware ezekre a végpontokra nem vonatkozik; kizárólag a Szalon Beállítások és Staff Management ágakon érvényesül.

METHOD	Útvonal	Leírás
GET	/api/provider/calendar/appointments	Lekéri az aktuális provider saját foglalásait. Opcionális startDate és endDate query paraméterekkel dátumintervallumra szűrhető; a CalendarSection intervallummal hívja, míg az

		üzenetindítási folyamat paraméter nélkül használja a korábbi regisztrált ügyfelek kigyűjtésére.
POST	/api/provider/calendar/appointments	Manuális foglalás feltöltése a naptárba, kiválasztva egy specifikus szolgáltatást és időpontot.
GET	/api/provider/calendar/fully-booked-days	Speciális végpont, amely végigmegy egy adott intervallum napjain és az availabilityService.js segítségével kiszámolja, mely napokon a Provider legrövidebb szolgáltatásához sem helyezhető el további szabad idősáv (a vizsgálatot az availabilityService a nyitvatartás, a meglévő foglalások és az aktív provider_time_blocks figyelembevételével végzi).
GET	/api/provider/time-blocks/raw	Visszaadja a beállított szabadságokat/kieső időket beágyazott ismétlődés adatokkal az Availability táblázathoz.
POST	/api/provider/time-blocks	Új szünet bejegyzése; backend szinten ellenőrzi azonnal az ütközéseket (conflicts) a lefoglalt vendégekkel.
PUT / DELETE	/api/provider/time-blocks/:id	Egyszeri szünet, illetve ismétlődő szünet módosítása vagy törlése. Ismétlődő blokkoknál a PUT kérés body-ban kapott targetInstance értéke alapján módosíthat egyetlen példányt ('single') vagy a teljes sorozatot ('all'). DELETE esetén a targetInstance és instanceDate query paraméterek döntik el, hogy csak egy adott ismétlődő

		előfordulás, vagy a teljes blokk törlődik.
GET	/api/provider/calendar/appointments/:id	Egyetlen, a bejelentkezett providerhez tartozó foglalás részleteinek lekérése. A backend elutasítja, ha a foglalás nem a hívó provider ID-jához tartozik.
DELETE	/api/provider/calendar/appointments/:id	Véglegesen (hard) törli a foglalást a appointments táblából (DELETE FROM appointments WHERE id = ?). Ennek a végpontnak a háttérfolyamatai: (1) sendAppointmentCancellation aszinkron e-mail a regisztrált ügyfélnek, (2) notifyWaitlistForCancelledSlot aszinkron értesítés a felszabadult időpontra várakozóknak.
GET	/api/provider/calendar/working-hours	A bejelentkezett Provider szalonjának nyitva-/zárási órájának lekérése (AvailabilityManagement és CalendarSection használja a manuális foglalások és szünetek időkorlát-validációjához).
PATCH	/api/provider/calendar/appointments/:id/status	Módosítja a foglalás státuszát a scheduled, completed, canceled, no_show értékek egyikére (tulajdonjog-ellenőrzéssel; a deleted érték ezen a végponton nem állítható).
GET	/api/provider/time-blocks?startDate&endDate	expanded, dátumtartományra bontott time block példányok listázása naptárnézethez.

Működési logika és Adatfolyam:

- **1. Napi/Heti foglalások kezelése (Appointments):**

- **Betöltés és inicializálás:** Amint megnyílik a CalendarSection komponens, egy sor független, aszinkron lekérés hajtódik végre a useEffect segítségével. A rendszer letölti a nyitvatartást (working-hours), az eddigi foglalásokat (appointments), és memóriába emeli az aktív szolgáltatások listáját (services state). Erre a manuális foglalási űrlap szolgáltatás-választójánál van szükség.
- **Új manuális foglalás:** Ha a szolgáltató új vendéget rögzít, a CreateAppointmentModal meghívásával egy űrlap nyílik meg. A kiválasztott szolgáltatás és időpont alapján a kliens POST /api/provider/calendar/appointments kérést küld a szervernek. A backend a POST /appointments során több validációt is futtat: ellenőrzi, hogy az időpont a szalon nyitvatartási idején belül van-e (opening_hours és closing_hours alapján), nem ütközik-e más foglalással (checkProviderAppointmentConflicts), és nem esik-e a Provider beállított szünetébe (getExpandedTimeBlocksForDate). Ütközés esetén 409-cel válaszol. Ha a foglalás regisztrált felhasználóhoz (nem guest) szól, és a megadott e-mail nem található a users táblában, a backend 404 Not Found hibával válaszol (üzenet: „Nincs ilyen email címhez regisztrált felhasználó"), és nem hozza létre automatikusan a fiókot, a Providernek vendégfoglalásként (is_guest: true) kell rögzítenie az ilyen ügyfelet.
- **Foglalások részletezése, chat indítása:** Egy már létező naptári bejegyzésre kattintva az AppointmentDetailModal jelenik meg. A komponens dinamikusan színezi a státuszokat (pl. zöld a scheduled, szürke a completed). Emellett beépített logikájával (onOpenChat) azonnali relációt teremt az adott felhasználó profiljával, megnyitva a chat ablakot.

- **2. Elérhetőségek és szabadidők menedzsmentje (Time Blocks):**
  - **Szünet (TimeBlock) szerkesztési folyamata:** Ha a Provider kieső időt (pl. ebédszünet vagy heti szabadnap) szeretne rögzíteni, a TimeBlockModal-t használja. Mentés előtt a frontenden futó validate() rutin szigorúan ellenőrzi az adatokat (pl. az endTime későbbi-e a startTime-nál, érvényesek-e a kiválasztott ismétlődési napok).
  - **Többnapos szabadság ("Multiday") bevitele:** Amikor napokon átívelő (mode === 'multiday') szabadságot rögzít a szolgáltató, a handleSave egy kliensoldali ciklusban a kiválasztott időszak minden napjára külön POST /api/provider/time-blocks kérést indít. Így minden naphoz önálló rekord kerül a provider_time_blocks táblába, ami egyszerűsíti az utólagos szerkesztést és törlést.
  - **API kommunikáció és ütközéskezelés (Conflicts):** Az új szünet (time block) beküldésekor a POST /api/provider/time-blocks végpontot kiszolgáló backend timeBlocksApi.js validálja az időintervallumot, majd az appointmentQueries.checkAppointmentConflicts(...) segítségével ellenőrzi, hogy esik-e aktív foglalás az új szünet időszakába. Ütközés esetén 409 Conflict választ ad conflicts listával, és nem hozza létre az időblokkot. A React UI ilyenkor figyelmeztető toast értesítést jelenít meg a szolgáltatónak, hogy intézkedni tudjon az ütköző ügyfelek felé.
    - A POST /api/provider/time-blocks átfedő nem ismétlődő blokkoknál a válasz `merged` boolean mezőt és szöveges üzenetet ad vissza. Az új

blokk legfeljebb 30 napos lehet (nem ismétlődő esetben), ennél nagyobb tartamot a backend 400-as hibával elutasít.

    - Az ismétlődő szünet egyetlen példányának módosítása (PUT /api/provider/time-blocks/:id targetInstance: 'single' payloaddal) speciális kezelést kap: a backend lezárja a meglévő ismétlődő sorozatot a kérdéses nap előtti napon (endRecurringBlockAt), létrehoz egy új, egyszeri (non-recurring) blokkot az adott napra, majd a sorozatot újraindítja a következő naptól az eredeti recurrence_pattern és recurrence_days megőrzésével.
    - Az ismétlődő szünet egyetlen példányának törlése hasonló sorozatbontási logikával működik. A frontend a DELETE /api/provider/time-blocks/:id?targetInstance=single&instanceDate=YYYY-MM-DD hívást küldi el. A backend ilyenkor lezárja az eredeti ismétlődő sorozatot a törölt nap előtti dátummal, majd szükség esetén új ismétlődő blokkot hoz létre a következő naptól az eredeti recurrence_pattern és recurrence_days megőrzésével. Így nem a teljes ismétlődő szabály törlődik, hanem csak az adott kiválasztott előfordulás esik ki.
  - **Vizuális prevenció:** A rendszer kifejezett figyelmet fordít a hibamegelőzésre. Ha a backend /fully-booked-days API válasza szerint egy napon már nem maradt szabad időrés, a UI azt a napot vizuális indikátorral jeleníti meg.

Vendégkör kezelése (Customer Management)

Leírás:

A szolgáltatók ezen a felületen követhetik nyomon azokat a vendégeket, akik már sikeresen lefoglaltak legalább egy időpontot náluk (alkalmazáson keresztül regisztrált felhasználók, és a szolgáltató által manuálisan naptárba rögzített "vendégek" egyaránt). A modul felül egy statisztikai sávban összegzi a legfontosabb metrikákat (összes vendég, visszatérési arány, legnépszerűbb szolgáltatások). A lista memóriában kereshető, az egyes ügyfelekre kattintva pedig egy részletes oldalpanel (Drawer) nyílik meg. Innen közvetlen áttekintést kapunk a múltbéli foglalásokról, regisztrált ügyfél esetén beszélgetést (Chat) kezdeményezhetünk, vagy egyetlen kattintással hivatalos e-mail emlékeztetőt küldhetünk nekik.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **CustomersSection.jsx:** A fő konténer. Lekéri az adatokat egy párhuzamos hívás keretein belül (Promise.all), kezeli a keresősáv helyi logikáját és menedzseli, hogy melyik vendég van épp kiválasztva (ez is nyitja meg a Drawert).
  - **CustomerStatsStrip.jsx:** Dedikált vizuális komponens, ami megjeleníti a backendtől kapott aggregált (számított) statisztikákat (összes vendég, visszatérési arány, top szolgáltatások).

- **CustomerListItem.jsx:** Az egyes vendégeket reprezentáló interaktív sor a listában.
- **CustomerDetailDrawer.jsx:** Egy "Drawer" (képernyő széléről becsúszó vizuális panel), ami megnyitáskor egy konkrét vendég részletes foglalási előzményeit tölti le és jeleníti meg, dinamikusan lekezelve a "regisztrált" vs. "guest" felhasználók közötti információs különbségeket. A drawer a regisztrált ügyfél legutóbbi, adott providerre vonatkozó értékelését jeleníti meg, ha van ilyen.

- **Adatbázis táblák (MySQL):**
  - **appointments:** A foglalások fő táblázata. Mivel nincs klasszikus izolált "customer" kapcsolótábla a Bookly-ban, a backend a foglalások táblájából aggregálja az egyedi ügyfeleket, amiből kiszámolja a statisztikákat (visszatérési arány) és kilistázza őket (a vendégkör a bejelentkezett provider saját foglalásaiból aggregálódik. A registered és guest részletes nézetek az adott providerhez kötött foglalási előzményeket adják vissza.).
  - **users:** Tartalmazza a regisztrált Bookly felhasználók adatait, amelyekből a kliens neve, profilképe és elérhetőségei letöltésre kerülnek (a manuálisan hozzáadott "vendégek" csak a foglalás rekordjaiban léteznek emailként és névként).
  - **services:** a vendégstatisztikában megjelenő legnépszerűbb szolgáltatások és a foglalási előzmények szolgáltatásnevei innen kapcsolódnak az appointments.service_id mezőn keresztül.
  - **ratings:** regisztrált ügyfelek részletes nézeténél a legutóbbi aktív provider-értékelés innen kerül lekérésre (provider_rating, provider_comment, created_at), provider és user alapján szűrve.

API végpontok (backend/api/calendar/customerRoutes.js):

METHOD	Útvonal	Leírás
GET	/api/provider/calendar/customers/stats	Aggregált statisztika számítása és visszaküldése (összes ügyfél, visszatérők aránya %-ban, top szolgáltatások) a bejelentkezett Provider ID alapján.
GET	/api/provider/calendar/customers	Lekéri az összes vendég listáját a providerhez kötődően. A lista integráltan (merged) tartalmazza a Bookly rendes felhasználókat

		és az in-house rögzített vendégeket.
GET	/api/provider/calendar/customers/registered/:userId	Részletes nézethez lekéri az adott regisztrált felhasználó bejelentkezett providerhez kötött foglalási előzményeit.
GET	/api/provider/calendar/customers/guest?email=…	Kifejezetten regisztráció nélküli, vendég ügyfelek foglalási előzményeit gyűjti össze email cím azonosító alapján (URL query segítségével).
POST	/api/provider/calendar/customers/remind	Emlékeztető e-mailt küld a megadott vendégnek a szalon nevére hivatkozva; a userId (regisztrált) és a guestEmail (vendég) bemenetet egyaránt elfogadja.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés:**
  - Amikor a szolgáltató a "Vendégek" menübe lép, a useEffect egy aszinkron fetchData() rutint indít el. A maximális hálózati hatékonyság érdekében a kód a Promise.all paradigmát alkalmazza, amely párhuzamos lekérést indít a /stats és a /customers végpontokat, így csökkentve a lekérési és válaszidőt. A válaszokat a React state-ek (stats, customers) tárolják el.
- **Keresési és UI szűrési logika:**
  - A lista felett található keresősáv nem generál új API hívásokat minden egyes billentyűleütésnél. Teljes mértékben lokális kliens-oldali szűrést alkalmaz (filteredCustomers megalkotása JavaScript filter metódussal). Ez folyamatosan validálja (kisbetűsítve) a találatokat név és email egyezés alapján a már memóriában tartott listán.
    - **Megjegyzés:** a backend GET /api/provider/calendar/customers végpontja támogat egy opcionális search query paramétert is (LIKE-mintával dolgozó SQL-szűrés), de a jelenlegi UI a teljes listát egyszer kéri le, és csak kliensoldalon szűr.
- **UI viselkedés (A Részletes Drawer panel):**

- Egy vendégre kattintva a rendszer beállítja a selectedCustomer state-et a komponensben. Ezzel feltételesen (React-portál segítségével) megjelenik a képernyő jobb oldaláról becsúszó CustomerDetailDrawer panel.
- Az oldalpanel (Drawer) DOM-ba történő betöltését követően egy saját belső useEffect indul el. Ez egy intelligens elágazással megvizsgálja a kapott property-t: ha a vendég manuális (customer.is_guest), akkor query paraméterrel a /guest?email=... végpontot, ha valós user, a /registered/:id végpontot kérdezi le, és azonnal kirendereli annak profilképét/kezdőbetűit (getInitials generátor) és az előzményeket.

- **Speciális logika (Hibrid Emlékeztető küldés):**
  - A "Remind" (Emlékeztető küldése) gomb szintén egy hibrid logikát fed. Megnyomásakor a handleRemind() funkció validálja, hogy rendes userről vagy guestről van-e szó.
  - Maga a POST payload eszerint dinamikusan idomul: body = customer.is_guest ? { guestEmail: customer.email } : { userId: customer.id }.
  - A backend szerver oldalán a JSON adatokból kinyeri az azonosítót, megkeresi hozzá az adatbázisban a kliens és a szalon pontos nevét (getReminderCustomerInfo), végül – leellenőrizve hogy a user rendelkezik-e email címmel – elindít egy aszinkron sendCustomerReminder folyamatot a backend Nodemailer-alapú SMTP emailService rétegén keresztül. Ennek végeztével a UI egy "Toast" (Sikeres értesítés) értesítést jelenít meg a szolgáltatónak.

Üzenetek és Kapcsolattartás (Messaging / Chat)

Leírás:

Ez a modul egy valós idejű (real-time) kommunikációs csatornát biztosít a szolgáltatók (Providers) és a regisztrált felhasználók (Users) között. A felület ketté van osztva egy bal oldali beszélgetés-listára és egy jobb oldali üzenetváltás nézetre (mobil részen ez dinamikusan váltakozik). A szolgáltató nemcsak válaszolni tud a vendégek megkereséseire, hanem a korábbi foglalások alapján proaktívan új beszélgetést is tud kezdeményezni. A rendszer Socket.io technológiát használ az azonnali kézbesítéshez, és azonnal nyomon követi az olvasatlan üzenetek számát.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **MessagesSection.jsx:** A fő konténer komponens, amely a Dashboardba simul. Inicializálja a Socket.io csatlakozást, tárolja a beszélgetések listáját (conversations state) és az aktív beszélgetés üzeneteit (messages state). Továbbítja a függőben lévő (pending) megnyitásokat, ha a vendégek listájából (Customer modultól) érkezett a felhasználó.

- **messagingService.js:** Egy dedikált szolgáltatásréteg, amely az összes REST API hívást aszinkron módon elvonatkoztatja a frontend komponensektől (getConversations, sendMessage, stb.).
- **ConversationList.jsx és MessageThread.jsx:** Újrahasználható UI komponensek, amelyek megjelenítik az üzenetbuborékokat és a beviteli mezőket, kezelve a "ki a küldő" (kliens vs. provider) logikát.

- **Adatbázis táblák (MySQL):**
  - **conversations:** A beszélgetéseket (szobákat) regisztráló tábla. Tárolja, hogy melyik provider_id és user_id van összekötve, valamint gyorsítótárazza a legutolsó üzenet kivonatát (last_message_preview), a küldés idejét (last_message_at) és az olvasatlan státuszokat (provider_unread_count, user_unread_count).
  - **messages:** A tényleges üzenetek táblája, amely egy conversation_id-hoz kapcsolódik. Eltárolja a text formátumú üzenetet (content), a küldő szerepkörét (sender_role: 'user' vagy 'provider') és annak egyedi azonosítóját (sender_id), opcionálisan a kapcsolódó foglalás azonosítóját (appointment_id, NULL-able), egy is_read logikai jelzőt és a létrehozási időbélyeget (created_at).

API végpontok (backend/api/messagingApi.js):

METHOD	Útvonal	Leírás
GET	/api/messages/conversations	Lekéri a felhasználóhoz (jelen esetben a providerhez) tartozó összes beszélgetési szobát a legutóbbi üzenetek előnézetével.
GET	/api/messages/conversations/:id	Egy specifikus beszélgetéshez letölti a tényleges üzenetek (chat-buborékok) kronologikus listáját.
POST	/api/messages/conversations	Létrehoz vagy visszaad egy létező beszélgetést a provider_id és user_id páros alapján. Customer felől a backend megköveteli, hogy legyen korábbi foglalás a célzott Providerrel (checkBookingExists); enélkül 403-mal elutasít. A frontend csak a saját korábbi vendégeit kínálja fel.
POST	/api/messages/conversations/:id/messages	Egy text üzenetet küld el egy létező beszélgetésbe. Azonnal frissíti a conversations.last_message_preview, last_message_at mezőket, és inkrementálja a címzett oldalának olvasatlan-számlálóját

		(provider_unread_count vagy user_unread_count).
PATCH	/api/messages/conversations/:id/read	Nullázza az olvasatlan üzenetek számát, miután a felhasználó megnyitotta a beszélgetést.
DELETE	/api/messages/conversations/:id	Törli a conversations rekordot; az ON DELETE CASCADE szabály miatt az összes kapcsolódó messages rekord is fizikailag törlődik. A művelet kétoldalú: a vendég oldalán is eltűnik a beszélgetés.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés:**
  - Amikor a MessagesSection betölt (+ mount), useEffect-ből egyrészt kikéri az összes meglévő beszélgetést a loadConversations() függvényen keresztül, másrészt inicializálja a WebSocket kapcsolatot.
  - A WebSocket (io(API_URL, { auth: { token } })) biztonságos, valós idejű hidat képez a szerverrel, JWT token alapon azonosítva a providert.
- **Szerkesztési logika (Üzenet megnyitása és olvasása):**
  - Egy beszélgetésre kattintva meghívódik a handleSelectConversation. Ez beállítja a selectedConversation-t aktívként, lekéri az ehhez tartozó korábbi üzeneteket, és rögtön küld egy PATCH /api/messages/conversations/:id/read REST kérést a backendnek.
  - Ilyenkor a komponens azonnal nullázza a provider_unread_count értékét a frontend kliens állapotában is, és jelzi a szülőkomponens számára (onUnreadChange prop), hogy csökkent az olvasatlan üzenetek száma a menürendszer (piros értesítési pálca) számára.
- **API kommunikáció és valós idejű kapcsolat:**
  - Amikor a szolgáltató elküld egy üzenetet (handleSend), a UI POST /api/messages/conversations/:id/messages kérést küld a szervernek. A küldés idejére egy betöltést jelző / spinner (sending state) UI elem jelenik meg az üzenetmezőben. Sikeres visszatéréskor a kliens lokális állapota is frissül: a setMessages hozzáfűzi az új üzenetet a beszélgetéshez, és a beszélgetéslistán az adott szoba utolsó üzenetének előnézete és időbélyege is felülíródik.
  - Amennyiben a vendég válaszol, a szerver a Socket.io-n keresztül közvetít egy new_message eventet. A frontend kliens elkapja a socket.on('new_message', …) eseménykezelővel (socketRef.current.on), dinamikusan hozzáfűzi az új üzenetet a megjelenített üzenetlistához, amennyiben a beszélgetés éppen aktív; ellenkező esetben a szoba olvasatlan-számlálóját inkrementálja eggyel.
  - A szerver oldalán a Socket.io kapcsolatra egy globális handshake-middleware fut: a kliens által adott JWT-t a szerver jsonwebtoken-nel verifikálja, és a

sikeresen csatlakozó socketet egy ${role}:${userId} mintázatú szobához (socket.join) csatlakoztatja. Az új üzenet eseményt a backend a messagingApi.js-ben a megfelelő szobába (io.to(...).emit('new_message', ...)) küldi, így a címzett kliens akkor is megkapja, ha másik beszélgetés van éppen aktívan nyitva.

- **Speciális logika (Új beszélgetés indítása múltbéli vendéggel):**
  - A Provider felületről proaktívan is lehetőség van beszélgetést indítani. Az „Új üzenet küldése" műveletnél a rendszer a GET /api/provider/calendar/appointments végponton lekéri a provider saját foglalásait, majd ezekből a user_id-val rendelkező, regisztrált ügyfeleket egyedi listává szűri. A guest-foglalások kimaradnak, mert chat csak regisztrált felhasználóval indítható.
  - Kiválasztás után a frontend a POST /api/messages/conversations végpontot hívja meg a provider_id és user_id párossal. Ha ehhez a pároshoz már létezik beszélgetés, a backend a meglévő szoba azonosítóját adja vissza; ha nem, új beszélgetést hoz létre, és a UI automatikusan arra fókuszál.
- **UI viselkedés (Mobile vs Desktop):**
  - Reszponzív logikával épül fel: Asztali képernyőn (md:flex) egymás mellett (split-screen) látszódik a lista és aktuális chat thread.
  - Mobilon (hidden md:flex kondíciókkal) a mobileView lokális state dönt (annak értéke list vagy thread), így kiküszöböli a túlzsúfolt kis kijelzőt egy tiszta vizuális lapozgatással.

Személyes Profil és Jelszókezelés (Provider Profile)

Leírás:

Míg a Szalon Menedzsment modul (3. pont) a globális szalon cégadatokat módosítja (amihez manager jog kell), ez a modul kizárólag az egyéni szolgáltató (Provider/Alkalmazott) személyes profiljára fókuszál. Itt minden fióktulajdonos – jogosultságtól függetlenül – szerkesztheti a saját, vendégek felé publikusan megjelenő nevét, privát telefonszámát, rövid bemutatkozását, illetve lecserélheti a személyes avatárképét (profile picture). Ebből a felületből érhető el a jelszócsere-mechanizmus (PasswordModal) is.

Érintett fájlok és Adatbázis táblák:

- **Frontend komponensek és hook-ok:**
  - **ProfileModal.jsx:** A Dashboard (Irányítópult) menüjéből ("Profilom") lenyíló komponens. Megjeleníti a személyes űrlapot, kezeli az avatár előnézetét/állapotát (pictureUploading state), és gátolja a csak olvasható mezők (pl. e-mail) módosítását. Segédlogikát tartalmaz a kezdőbetűk kigenerálásához (Initials), abban az esetben, ha nincs még kép feltöltve.
  - **PasswordModal.jsx:** Egy beágyazott al-modal (sub-modal), amely a ProfileModal-ból nyitható meg. Három inputot kezel egymás alatt: a biztonsági

validáció miatt kötelező "Jelenlegi jelszó" és az "Új jelszó", illetve "Új jelszó megerősítése" mezőket.

- **Adatbázis táblák (MySQL):**
  - **providers:** A modul a providers tábla name, phone, description, password_hash, profile_picture_url mezőit írja; az email csak olvasott. A teljes sémaleírást lásd a Regisztráció és Bejelentkezés szekciónál.

API végpontok (backend/api/salon/providerProfileRoutes.js):

Amikor ezt az API-t bekötik (salonApi.js), a prefix automatikusan /api/salon lesz a fő szerver fájlban. Minden végpontot az AuthMiddleware véd, amely automatikusan extrahálja az érintett Provider saját ID-ját a dekódolt JWT token-ből (request.user.userId), kiküszöbölve más profiljának módosítását.

METHOD	Útvonal	Leírás
GET	/api/salon/me	Bekéri a belépett provider összes releváns személyes adatát a Profile űrlap feltöltéséhez.
PUT	/api/salon/me	Elküldi egy JSON csomagként a felhasználó új név, leírás és telefon adatait, a validációs szabályok ellenőrzése után rögzíti a módosításokat az adatbázistáblában.
POST	/api/salon/me/picture	Képfájlt fogad (jpg, png, webp legfeljebb 5MB). Multipart FormData-val dolgozik. Multer middleware használatával elmenti a szerver /uploads/profiles/ mappájába.
PUT	/api/salon/me/password	A jelszócsere végpont. JSON body-ban várja a currentPassword, newPassword és confirmPassword mezőket. A backend bcrypt.compare segítségével ellenőrzi a jelenlegi jelszó helyességét, megköveteli, hogy newPassword.length >= 8 legyen, és hogy newPassword === confirmPassword teljesüljön. Bármelyik szabály megsértése esetén 400 Bad Request választ ad. Sikeres művelet esetén frissíti a providers.password_hash mezőt, törli a korábbi provider refresh tokeneket, új refresh tokent állít be HttpOnly cookie-ban, és az új access tokent az accessToken válaszmezőben adja vissza.

Működési logika és Adatfolyam:

- **Betöltés és jogosultság kezelés (Self-Scope):**

- Ellentétben a szalon alapú vagy munkatárs (staffRoutes) kezeléssel, ezeknél a hívásoknál nincs isManager kitétel. Bármelyik hitelesített szolgáltató lekérdezheti és frissítheti magát (mert a célzott provider azonosítója a dekódolt JWT payloadból származik, ezért a végpont mindig a bejelentkezett szolgáltató saját profilján dolgozik).

- **Szerkesztési logika (Profil):**
  - Amikor a felhasználó a Dashboard-on a lenyíló "Profilom" fülre kattint, a szülő komponens aszinkron GET hívást intéz, és a kapott adatokat betölti a modal profileFormData állapotába.
  - Mentéskor simple JSON kommunikáció történik a PUT végpont felé. Siker esetén a fő menü Navbar-ja is egyből reagál (pl. frissül az ott megjelenő név is a props-on keresztül).
- **Speciális logika (Avatar Multipart Képfeltöltés):**
  - Hasonlóan a Szalon Banner/logó cseréhez, a profilkép módosítás azonnal, a "Kép módosítása" inputra való (onChange={onPictureUpload}) kattintás után elküldi a fájlt a backend felé POST-tal (nem szükséges megvárni az űrlap végső Mentés gombjára kattintást).
  - Ez biztosítja, hogy a Backend Multer middleware-e feldolgozza a feltöltött fájlt, elmenti a könyvtárstruktúrában (Pl.: /uploads/profiles/provider-<providerId>-<timestamp>.jpg), és a visszaküldött publikus URL azonnal újra tudja renderelni a Provider képét a modal és a headerben egyaránt.
- **Jelszócsere (Biztonsági logika):**
  - A PasswordModal.jsx a PUT /me/password API felé egy zárt flow-t képvisel. Ha a backend bcrypt leellenőrzése (bcrypt.compare) a currentPassword-re hibát dob, azonnali 400-as statuskóddal blokkolja a műveletet (Helytelen a jelenlegi jelszó).
  - A jelszócsere hibáit és sikeres állapotát a modal saját passwordError és passwordSuccess állapotai kezelik; a felület ezek alapján jelenít meg felhasználói visszajelzést. Sikeres jelszócsere után a backend törli az adott fiók korábbi refresh tokenjeit, majd az aktuális kliens számára új access + refresh tokenpárt állít ki. Az új access token a JSON válasz accessToken mezőjében érkezik, az új refresh token pedig HttpOnly cookie-ként kerül beállításra. Emiatt az aktuális eszköz bejelentkezve marad, míg a korábbi refresh tokenek érvényüket vesztik.

Frontend Tesztelési Rendszer

Leírás

A Bookly frontend tesztelési rendszere Playwright alapú end-to-end tesztkészlet, amely a React webalkalmazás legfontosabb felhasználói folyamatait ellenőrzi valós böngészőkörnyezetben. A tesztek nem komponenseket izolálnak, hanem teljes UI-n keresztül végrehajtott user flow-kat vizsgálnak: oldalmegnyitás, űrlapkitöltés, kattintás, navigáció, modálkezelés, validáció és végállapot-ellenőrzés.

A rendszer célja, hogy automatikusan visszajelezze, működnek-e az alkalmazás kritikus frontend útvonalai ügyfél és provider oldalon. A tesztek lefedik az autentikációt, regisztrációt, foglalást, foglalás lemondását, profilkezelést, fióktörlési validációt, provider regisztrációt, szolgáltatáskezelést és naptárhasználatot.

A jelenlegi kód alapján a tesztkészlet 10 spec fájlban 33 explicit test(...) tesztesetet tartalmaz. A tesztek minden fontosabb állapotról screenshotot készítenek, így a futás eredménye nemcsak pass/fail szinten, hanem vizuálisan is visszakövethető.

Érintett fájlok és struktúra

/tests mappa

A **frontend tesztek a tests mappában találhatók**. A mappa önálló Playwright tesztkörnyezetként működik: saját konfigurációval, setup fájllal, tesztfájlokkal, helper réteggel és fixture-ökkel rendelkezik.

A Playwright konfigurációt a playwright.config.js tartalmazza. A tesztek a *.spec.js fájlokra szűrnek, Chromium projektben, Desktop Chrome beállítással és 1920x1080 viewporttal futnak. A konfiguráció baseURL értéke http://localhost:5173, a suite előtt lefuttatja a globális setupot, 2 workerrel dolgozik, egyszeri retry-t engedélyez, HTML és list riportot készít, valamint hibánál screenshotot, retry esetén trace-t, sikertelen futásnál videót ment.

globalSetup.js: egyszer fut le a teljes tesztsuite előtt. Ellenőrzi a backend elérhetőségét, majd idempotens módon előkészíti a tesztekhez szükséges adatokat: a demo ügyfelet, a demo providert és a hozzá tartozó Test Salon szalont. Az ügyfélprofilban telefonszámot is beállít, mert a foglalási folyamat ezt igényli.

package.json: a futtatási scripteket tartalmazza: normál tesztfuttatás, headed mód, riport megnyitása és Playwright codegen.

Tesztfájlok részletes bemutatása

- **Login.spec.js:** ügyfél login oldal betöltése, üres mezők validációja, hibás jelszó kezelése, sikeres bejelentkezés dashboardra. A sikeres teszt egyedi felhasználót generál és azzal lép be.
- **Register.spec.js:** ügyfél regisztrációs oldal, üres mezős validáció, eltérő jelszavak hibája, teljes regisztráció utáni login flow.

- **Booking.spec.js:** fő foglalási flow és múltbeli dátumok tiltása a foglalási naptárban. A fájl saját tesztjei serial módban futnak. A lemondási tesztek külön fájlban vannak, és minden esetben saját friss foglalást hoznak létre.
- **CancelAppointment.spec.js:** foglalás lemondási tesztek. Minden teszt saját friss foglalást hoz létre a foglalási helperrel, majd a modál megszakítását vagy a lemondás megerősítését ellenőrzi. Két tesztesetet tartalmaz: a lemondási modál bezárását mentés nélkül, illetve a megerősített lemondást és a sikerüzenet megjelenését.
- **CustomerProfile.spec.js:** profil fül betöltése, szerkesztési modál, telefonszám mentése, jelszómódosítás validációja.
- **CustomerAccountDeletion.spec.js:** fióktörlési modál megnyitása, bezárása és üres jelszóra adott validáció. Tényleges fióktörlést nem hajt végre.
- **ProviderLogin.spec.js:** provider login oldal, üres mezők, hibás adatok, sikeres bejelentkezés a /ProvDash felületre.
- **ProviderRegister.spec.js:** provider regisztráció választóképernyő, új szalon létrehozása, meglévő szalonhoz csatlakozás, kötelező mezők validációja, eltérő jelszavak ellenőrzése. A szalonkód validációt egy teszt page.route mockkal stabilizálja.
- **ProviderServiceCRUD.spec.js:** szolgáltatások szekció betöltése, új szolgáltatás modál, üres név validációja, időbélyeggel egyedivé tett szolgáltatás létrehozása.
- **ProviderCalendar.spec.js:** provider naptár betöltése, kattintható napcella kiválasztása.

/helpers mappa

A **helpers mappa 4 segédfájlt tartalmaz**. Ezek az ismétlődő bejelentkezési, adatgenerálási és foglalási lépéseket szervezik ki.

- **auth.js:** ügyfél login helper. Megnyitja a /login oldalt, kitölti az adatokat, kattint a bejelentkezésre, majd megvárja a /dashboard útvonalat.
- **providerAuth.js:** provider login helper. A /provider/login oldalon léptet be, majd a /ProvDash URL-re vár. Az alapértelmezett adatok környezeti változókkal felülírhatók.
- **testData.js:** időbélyeg-alapú egyedi tesztfelhasználót generál, főleg regisztrációs és login flow-khoz.
- **booking.js:** a foglalási varázsló közös logikája. Keresést indít, szalont nyit, providert és szolgáltatást választ, időpontot keres, majd a foglalási összefoglalóig navigál. Ha nincs szabad slot, false értékkel jelzi, hogy a teszt skipelhető.

/fixtures mappa

A **fixtures mappa 2 fájlból áll**, és előkészített bejelentkezett állapotokat ad a teszteknek.

- **authFixture.js:** loggedInPage fixture-t definiál. Ez a normál Playwright page objektumot ügyfélként belépteti, majd a tesztnek már bejelentkezett oldalt ad át.

- **providerAuthFixture.js:** providerPage fixture-t definiál. Ez providerként léptet be, és a provider dashboard állapotból indítja a szolgáltatáskezelési és naptár teszteket.

A fixture-ök fő szerepe a közös state és setup kezelés. A tesztfájlokból így azonnal látszik, hogy egy teszt milyen előfeltétellel indul. Ha a teszt loggedInPage paramétert használ, akkor ügyfélként bejelentkezett állapotból fut. Ha providerPage paramétert használ, akkor providerként bejelentkezett állapotból fut.

Tesztek felépítése és működési logika

- **Playwright teszt felépítése**
  - A tesztek közös mintát követnek:
    1. A teszt URL-ről vagy fixture-ből indul
    2. Megvárja a szükséges UI állapotot
    3. Felhasználói műveleteket hajt végre
    4. expect assertionökkel ellenőrzi az eredményt
    5. Screenshotot készít a fontos állapotokról
  - A lokátorok főként getByRole, getByText, getByPlaceholder, getByLabel és célzott CSS selectorok. Az ellenőrzések nem belső React state-re, hanem látható UI-ra, URL-re, modálokra, mezőkre, toastokra és gombállapotokra épülnek.

- **beforeEach és setup logika**
  - A login és provider login spec fájlok beforeEach blokkal minden teszt előtt a megfelelő belépési oldalra navigálnak. A provider regisztrációs tesztek szintén közös beforeEach segítségével indulnak a /provider/register oldalról.
  - A bejelentkezést igénylő tesztek fixture-öket használnak. Így a foglalási, profil és fióktörlési tesztek loggedInPage állapotból, a provider dashboard tesztek pedig providerPage állapotból indulnak. A globális setup csak egyszer fut, és az alap tesztadatokat készíti elő.

- **User interakciók kezelése**
  - A tesztek valódi böngészős műveleteket hajtanak végre: goto, fill, click, waitForURL, waitForLoadState, modálnyitás, tabváltás és űrlapbeküldés. A saját alkalmazásvalidáció ellenőrzéséhez több teszt kikapcsolja a HTML natív validációt a form noValidate beállításával.
  - A provider regisztráció egyik tesztje page.route mockot használ a szalonkód validációjához. Ezzel a teszt célzottan a frontend lépésváltást és jelszóvalidációt ellenőrzi, nem a backend kódellenőrzéstől függ.

- **Assertion-ök működése**

- Az assertionök a felhasználó számára érzékelhető eredményekre fókuszálnak. Tipikus ellenőrzések:
  - megfelelő URL-re navigálás: /login, /dashboard, /ProvDash
  - kötelező mezős és hibás jelszavas validációk
  - sikeres regisztráció és bejelentkezés visszajelzése
  - foglalási sikerállapot
  - lemondási sikerüzenet
  - profilmentés visszajelzése
  - eltérő jelszavak hibája
  - létrehozott provider szolgáltatás megjelenése

- **Teljes user flow teszt felépítése**
  - A legösszetettebb flow a foglalás. A teszt bejelentkezett ügyfélként keresést indít, megnyitja az első szalont, elindítja a foglalási varázslót, kiválasztja a munkatársat és szolgáltatást, majd a naptárban elérhető időpontot keres. A helper több napot és legfeljebb 3 hónapot próbál végig, mert a foglalási helperben a maxMonthsToTry értéke 3. Sikeres slotválasztás után a teszt eljut az összefoglalóra, véglegesíti a foglalást, ellenőrzi a sikeres állapotot, majd megnyitja a foglalásaim nézetet.
  - Ez a flow egyszerre validálja a frontend routingot, a backend válaszokra épülő UI állapotokat, a naptárlogikát, a foglalási varázsló lépéseit és a sikeres végállapotot.

Screenshot mechanizmus

- **Mikor készül screenshot**
  - Screenshot akkor készül, amikor a felület fontos, dokumentálható állapotba kerül. Ilyenek például az oldalbetöltések, validációs hibák, kitöltött űrlapok, sikeres login vagy regisztráció, foglalási lépések, lemondási modálok, profilmodál, provider regisztrációs lépések, szolgáltatáslétrehozás és provider naptárnézet.
  - A screenshotok funkcionális almappákba kerülnek, például screenshots/login, screenshots/register, screenshots/booking, screenshots/customer-profile, screenshots/customer-account-deletion, screenshots/provider-login, screenshots/provider-register, screenshots/provider-services és screenshots/provider-calendar.

- **Hogyan van implementálva**
  - A screenshotok többsége közvetlen page.screenshot(...) hívás a spec fájlokban. A teszt előbb létrehozza vagy megvárja az állapotot, majd fájlútvonallal menti a képet.

- A foglalási helper opcionális screenshotPrefix paramétert is kezel. Ha a hívó teszt megadja, a helper a foglalási varázsló köztes lépéseinél automatikusan screenshotot készít. Emellett a Playwright konfiguráció hibánál automatikus screenshotot, retry esetén trace-t, sikertelen futásnál videót ment.

- **Milyen célt szolgál**
  - A screenshotok debug és dokumentációs célúak. Hibánál gyorsan megmutatják, milyen UI állapotban állt meg a teszt, sikeres futásnál pedig vizuálisan visszakövethetővé teszik a flow lépéseit. A számozott fájlnevek és funkcionális mappák miatt a folyamatok sorrendje utólag is értelmezhető.

Helper és Fixture architektúra

- **Duplikáció csökkentése**
  - A helper és fixture réteg kiszervezi a gyakran ismétlődő technikai lépéseket: login, provider login, tesztadat-generálás és a foglalási varázsló közös navigációja. Ennek köszönhetően a spec fájlokban az üzleti cél marad látható, a részletes navigációs és setup logika pedig központi helyen kezelhető.

- **Függvénytípusok**
  - A helper függvények három fő kategóriába sorolhatók:
    - autentikációs helper: ügyfél és provider login
    - tesztadat helper: egyedi felhasználó generálása
    - flow helper: több képernyős foglalási folyamat
  - Ez a felosztás támogatja az újrafelhasználást, és csökkenti annak esélyét, hogy ugyanaz a logika több tesztben eltérően legyen megírva.

- **Fixture-ök meghívása a tesztekből**
  - A fixture-t használó spec fájlok nem közvetlenül az @playwright/test csomagból importálják a test objektumot, hanem a megfelelő fixture fájlból. Ügyféloldalon a teszt loggedInPage: page paraméterrel indul, provider oldalon providerPage: page paraméterrel.
  - A teszt belül továbbra is page néven használja az objektumot, de az már előkészített, bejelentkezett állapotot jelent. Ez olvashatóbbá teszi a teszteket. Új bejelentkezett szerepkör vagy előkészített oldalállapot esetén ugyanez a fixture minta tovább bővíthető.

Működési logika és teszt adatfolyam

- **Teszt indítása → setup → user flow → assertion → screenshot**

- A futás a Playwright konfiguráció alapján indul. A globalSetup.js előkészíti az alap ügyfél- és provider adatokat, majd a Playwright betölti a spec fájlokat. A teszt indulhat közvetlen URL-ről, beforeEach blokkból vagy fixture által előkészített állapotból.
- Ezután a teszt végrehajtja a user flow-t, assertionökkel ellenőrzi az elvárt UI állapotot, majd screenshotot készít. Ha a flow külső feltételtől függ, például elérhető időponttól vagy kattintható naptári naptól, a teszt kontrolláltan skip állapotba kerülhet.

- **Adatok áramlása a fixture-ökön keresztül**
  - Az ügyféloldali flow alapja a test@test.com demo fiók, amelyet a globális setup biztosít. A loggedInPage fixture ezzel a fiókkal léptet be, majd a bejelentkezett oldalt adja tovább a foglalási, profil és fióktörlési teszteknek.
  - A provider oldali flow alapja a provider@test.com fiók és a Test Salon. A providerPage fixture providerként jelentkezik be, majd innen indulnak a szolgáltatáskezelési és naptár tesztek.
  - Az ügyféloldali sikeres regisztrációs és login flow-k a generateUser helperrel hoznak létre egyedi tesztfelhasználót, így elkerülhetők az e-mail cím ütközések ismételt futtatáskor. A provider regisztrációs tesztek nem ezt a helpert használják, hanem fix mezőértékekkel és mockolt szalonkód-validációval ellenőrzik a több lépéses regisztrációs folyamatot.

- **Helper-ek szerepe a folyamatban**
  - A login helper-eket elsősorban a fixture-ök hívják, a booking helper-t közvetlenül a foglalási és lemondási tesztek használják. A lemondási tesztek minden esetben saját új foglalást hoznak létre a booking helperrel, majd azon ellenőrzik a lemondási UI-t, ezért nem függnek előre meglévő foglalásoktól és a foglalási spec futási sorrendjétől sem.
  - A provider regisztrációs mockolt API válasza szintén stabilitási elem: a teszt a frontend lépésváltásra és validációra koncentrál, nem a backend szalonkód-adatállapotára.

Összegzés

A Bookly frontend tesztrendszere Playwright alapú, valós böngészőben futó end-to-end tesztkészlet. A legfontosabb ügyfél- és provider oldali folyamatokat ellenőrzi: autentikáció, regisztráció, foglalás, lemondás, profilkezelés, fióktörlési validáció, provider regisztráció, szolgáltatáskezelés és naptárhasználat.

A tesztek fő értéke, hogy a felhasználói felületen keresztül validálják az alkalmazás működését. Ez egyszerre érinti a UI-t, routingot, űrlapvalidációt, aszinkron betöltéseket és backenddel való együttműködést.

A helper és fixture architektúra csökkenti a duplikációt, központosítja a közös logikát, és olvashatóbbá teszi a spec fájlokat. A screenshot mechanizmus fejlesztői hibakereséshez és vizuális dokumentációhoz is hasznos.

A jelenlegi lefedettség nem teljes komponensszintű vagy vizuális regressziós tesztkészlet, de erős biztonsági hálót ad a projekt üzletileg kritikus frontend flow-jaira. A struktúra jól bővíthető új spec fájlokkal, helper függvényekkel vagy fixture-ökkel.

Backend Tesztelési Rendszer

Leírás

A Bookly backend tesztelési rendszere Jest alapú tesztkészlet, amely a Node.js/Express backend kritikus API útvonalait, middleware-eit és üzleti logikáit ellenőrzi. A tesztek célja annak validálása, hogy az autentikáció, foglaláskezelés, profilkezelés, keresés, provider működés és lokációs logika stabilan működik kontrollált bemenetek mellett.

A backend tesztek két fő szinten működnek:

- **integrációs tesztek:** Express routereket tesztelnek Supertest segítségével, mockolt adatbázis- és service réteggel;
- **unit tesztek:** önálló függvényeket vizsgálnak, például lokációszámítást, geokódolást és időblokk-kiterjesztést.

A tesztek nem valós adatbázist használnak, hanem jest.mock segítségével kontrollált válaszokat adnak az SQL és service moduloknak. Ez gyorsabbá, determinisztikussá és izoláltabbá teszi a futást. A jelenlegi kód alapján a backend tesztrendszer 8 .test.js tesztfájlból és 1 setup helper fájlból áll, összesen 110 explicit test(...) / it(...) tesztesettel.

Érintett fájlok és struktúra

/backend/tests mappa

A backend tesztek a tests mappában találhatók. A struktúra három fő részre oszlik:

- **integration:** API endpointok és middleware-ek Supertest alapú tesztjei;
- **unit:** önálló backend függvények unit tesztjei;
- **setup:** közös tesztsegédek, főleg JWT tokenekhez és tesztkörnyezeti változókhoz.

A Jest konfigurációt a jest.config.js tartalmazza. A tesztek Node környezetben futnak, a **/tests/**/*.test.js mintára illeszkedő fájlokat veszik figyelembe, és coverage riportot készítenek a coverage mappába. A coverage gyűjtés az api, services és sql könyvtárakra van beállítva.

A futtatást a package.json scriptjei kezelik. A fő parancs a npm test, amely jest --coverage futtatást indít. Emellett van watch mód is npm run test:watch paranccsal.

Tesztfájlok részletes bemutatása

- **<u>auth.test.js</u>:** ügyfél autentikációs útvonalak tesztelése. Lefedi a regisztrációt, login folyamatot, refresh token kezelést és logout működést. Ellenőrzi a hiányzó mezőket, rövid jelszót, duplikált e-mailt, hibás belépést, tiltott vagy törölt felhasználót, sikeres tokenkiadást és cookie-kezelést.
- **<u>authMiddleware.test.js</u>:** az AuthMiddleware működését ellenőrzi. Teszteli a hiányzó tokent, lejárt tokent, érvénytelen aláírást, tiltott customer/provider fiókot, törölt customer fiókot, nem létező usert, valamint az aktív customer és provider átengedését.

- **provAuth.test.js:** provider autentikációs és regisztrációs API-k tesztje. Lefedi a szalonkód validációt, provider regisztrációt meglévő szalonhoz csatlakozással, új szalon létrehozásával, tranzakciós rollbackkel duplikált adatoknál, valamint provider login hibákat és sikeres belépést.
- **user.test.js:** customer oldali API-k tesztje. Lefedi a profil lekérdezését és frissítését, jelszócserét, ügyféloldali foglalás létrehozását, foglalás lemondását, értékelés leadását és fióktörlést. A tesztek ellenőrzik a jogosultságot, validációkat, konfliktusokat, tranzakciós foglalási logikát és sikeres állapotokat.
- **calendar.test.js:** provider naptár és szolgáltatáskezelés API tesztje. Lefedi a dashboard statisztikákat, időpontlistát, provider oldali foglalás létrehozását vendégként és regisztrált userként, időpontlemondást, szolgáltatás létrehozását és törlését. Vizsgálja a szolgáltatás tulajdonlását, nyitvatartási időt, időpontütközést és aktív foglalások miatti törlési tiltást.
- **search.test.js:** keresési API tesztek. Lefedi a közeli szalonok sikeres keresését koordináták alapján, a hiányzó vagy hibás koordináták kezelését, a helynév geokódolási hibáját, a szolgáltatásnév szerinti szűrést, a szalon részlet lekérdezést és a szalontípusok listázását.
- **locationService.test.js:** lokációs service unit tesztjei. Ellenőrzi a távolságszámítást, közeli szalonok szűrését és rendezését, hiányzó koordináták kizárását, geokódolást, reverse geocodingot és cím autocomplete működést. A külső fetch hívások mockolva vannak.
- **expandTimeBlocks.test.js:** az időblokk-kiterjesztő logika unit tesztje. Lefedi az egyszeri blokkok szűrését, napi ismétlődést, heti ismétlődést, recurrence end date kezelését és üres inputot.

/setup mappa

- **testHelpers.js:** közös JWT helper funkciókat tartalmaz. Beállítja a teszt JWT secret értékeket, valamint access token, refresh token és lejárt access token generálására ad függvényeket. Ezeket főleg az autentikációs, middleware, user és calendar tesztek használják.

Tesztek felépítése és működési logika

Jest / Supertest teszt felépítése

Az integrációs tesztek közös mintát követnek:

1. A teszt mockolja az érintett SQL és service modulokat
2. Létrehoz egy minimális Express alkalmazást
3. Felcsatolja a vizsgált routert egy adott prefix alá
4. Supertesttel HTTP kérést küld az endpointnak
5. Beállítja a szükséges body-t, cookie-t vagy Authorization headert
6. Ellenőrzi a HTTP státuszt, response body-t és mellékhatásokat

A tesztek nem a teljes szervert indítják el, hanem csak az adott routerhez szükséges minimális Express appot építik fel. Ez gyorsabb, fókuszáltabb tesztelést ad, miközben az endpointok valódi request/response viselkedését ellenőrzi.

beforeAll, beforeEach és setup logika

A beforeAll blokkok jellemzően a tesztkörnyezet egyszeri előkészítését végzik: beállítják a JWT secret változókat, létrehozzák az Express appot, felcsatolják a routert, és tokeneket generálnak customer vagy provider szerepkörhöz.

A beforeEach blokkok minden teszt előtt alaphelyzetbe állítják a mockokat. A legtöbb fájl jest.clearAllMocks() hívást használ, a tranzakciós teszteknél pedig a mockolt connection metódusai is újra beállításra kerülnek. Ez megakadályozza, hogy egyik teszt mock állapota átszivárogjon a következőbe.

A unit tesztekben a beforeEach főleg a külső fetch mockolására szolgál, például geokódolási és autocomplete teszteknél.

Mockolás és izoláció

A backend tesztek egyik legfontosabb eleme a mockolt függőségi réteg. Az adatbázis, e-mail küldés, lokációs service, waitlist service és upload middleware több tesztben is mockolva van.

A mockolás célja:

- ne kelljen valós adatbázis-kapcsolat
- determinisztikus legyen minden válasz
- külön tesztelhető legyen az API logika
- ne történjen fájlrendszer-művelet az upload middleware-t importáló route-tesztekben
- ne menjen ki valódi e-mail vagy külső hálózati kérés
- könnyen szimulálhatók legyenek hibák, ütközések és edge case-ek

A tranzakciós működést igénylő tesztek mockolt connection objektumot használnak beginTransaction, query, commit, rollback és release metódusokkal. Ez különösen a provider regisztrációnál és az ügyféloldali foglalás létrehozásánál fontos. A fióktörlési teszt nem connection-tranzakciót mockol, hanem pool.execute hívásokat és a deleteUser függvényt.

Assertion-ök működése

Az assertionök főként az API szerződésre fókuszálnak:

- HTTP státuszkódok: 200, 201, 400, 401, 403, 404, 409, 422
- success mező értéke
- tokenek megléte
- cookie beállítása vagy törlése
- validációs hibák
- jogosultsági tiltások
- létrehozott erőforrás azonosítója
- visszaadott user, provider, salon vagy appointment adatok
- mockolt függvények meghívása vagy tranzakciós rollback

A tesztek így nemcsak azt vizsgálják, hogy egy endpoint “lefut-e”, hanem azt is, hogy a megfelelő üzleti és biztonsági válaszokat adja-e.

Teljes backend flow teszt felépítése

A backend flow-k tipikusan request-response alapon épülnek fel. Például egy ügyféloldali foglalás teszt logikája:

1. Customer access token létrehozása
2. AuthMiddleware státuszlekérdezés mockolása aktív userrel
3. Szolgáltatás, provider és szalon lekérdezésének mockolása
4. Tranzakciós connection előkészítése
5. User napi foglalásütközés és provider slotütközés ellenőrzésének mockolása
6. Appointment insert mockolása
7. Végső SELECT válasz mockolása
8. POST /user/appointments kérés küldése Supertesttel
9. 201 státusz, success: true és létrehozott appointment adatok ellenőrzése

A provider oldali naptár foglalási flow hasonló: tokennel indul, provider jogosultságot ellenőriz, szolgáltatást keres, nyitvatartást és foglalási ütközést validál, majd vendég vagy regisztrált user foglalást hoz létre.

Riport és coverage mechanizmus

Mikor keletkezik riport

A backend tesztek futtatásakor a Jest coverage riportot készít. A npm test script jest --coverage parancsot futtat, ezért a tesztek eredménye mellett lefedettségi információ is keletkezik.
A coverage célterületei a konfiguráció alapján:

- api/**/*.js
- services/**/*.js
- sql/**/*.js

A riport a coverage könyvtárba kerül.

Hogyan van implementálva

A coverage működését a Jest konfiguráció szabályozza. A jest.config.js Node tesztkörnyezetet használ, a tesztfájlokat a tests mappából gyűjti, a lefedettségi outputot pedig coverageDirectory: 'coverage' beállítással menti.

A backend tesztek nem készítenek screenshotot, mert nem böngészős UI-t tesztelnek. A dokumentációs és debug információt itt a Jest output, a státuszkód/assertion hibák, a mock hívási sorrendek és a coverage riport adják.

Milyen célt szolgál

A riport és coverage mechanizmus fejlesztői visszajelzést ad arról, hogy a backend API-k, service-ek és SQL segédlogikák mely részei vannak tesztekkel érintve. Hibás teszt esetén a Jest pontosan megmutatja, melyik assertion bukott el, milyen értéket kapott a teszt, és melyik fájlban történt a hiba.

Ez backend oldalon a frontend screenshotok szerepének megfelelője: nem vizuális állapotot rögzít, hanem futási, assertion és lefedettségi információt ad.

Helper architektúra

Duplikáció csökkentése

A backend tesztekben nincs Playwright-szerű fixture rendszer, de van közös helper és ismétlődő setup minta. A testHelpers.js kiszervezi a JWT környezet és token generálás logikáját, így nem kell minden tesztfájlban külön secretet és tokent kézzel felépíteni.
A routertesztekben visszatérő minta a buildApp() vagy egyszeri Express app létrehozás, a router felcsatolása, majd Supertest request indítása. A jogosultságot igénylő fájlokban helper jellegű lokális függvények is vannak, például customer vagy provider auth státusz mockolására.

Függvénytípusok

A backend tesztsegédek és ismétlődő minták fő típusai:

- JWT helper: access, refresh és expired token generálása
- környezeti helper: teszt JWT secret értékek beállítása
- Express app builder: minimális app létrehozása egy adott routerrel
- auth mock helper: aktív customer/provider státusz szimulálása
- tranzakciós mock: connection objektum beginTransaction, query, commit, rollback, release metódusokkal
- külső service mock: e-mail, lokáció, waitlist, upload és fetch izolálása

Ezek együtt biztosítják, hogy a tesztek rövidek maradjanak, és minden teszt csak a saját endpointjának viselkedésére fókuszáljon.

Setup helper használata a tesztekből

Backend oldalon nincs külön fixture fájl. A setup helper-t az autentikációt vagy JWT-t igénylő integrációs tesztek használják; ezekben a szükséges függvényeket importálják, és a setTestEnv() jellemzően beforeAll alatt fut. A search és unit tesztek nem minden esetben használják ezt a helper fájlt.

Az access tokeneket a tesztek szerepkör szerint generálják. Customer API teszteknél customer token, provider naptár teszteknél provider token készül; az authMiddleware tesztek pedig külön customer és provider tokeneket is generálnak a middleware ágak ellenőrzéséhez. A requestekben ezek Authorization: Bearer ... headerként jelennek meg, így az AuthMiddleware valós tokenfeldolgozási útvonalon fut, miközben az adatbázis státuszlekérdezése mockolt.

Működési logika és teszt adatfolyam

Teszt indítása → setup → API flow → assertion → riport

A backend tesztfutás a Jest konfiguráció alapján indul. A Jest betölti a tests mappában lévő .test.js fájlokat, előkészíti a mockokat, majd fájlonként lefuttatja a beforeAll és beforeEach blokkokat.

Egy tipikus integrációs adatfolyam:

1. Jest mockolja az adatbázis- és service modulokat
2. Az autentikációt vagy tokent igénylő tesztekben a setup helper beállítja a JWT secret értékeket
3. A teszt létrehoz egy Express appot a vizsgált routerrel
4. A teszt beállítja a mockolt adatbázisválaszokat
5. Supertest elküldi a HTTP kérést
6. A router és middleware feldolgozza a kérést
7. A teszt ellenőrzi a státuszkódot, body-t, cookie-t vagy mock hívást
8. Jest riportolja az eredményt és coverage adatot gyűjt

Adatok áramlása a helper-eken keresztül

Az autentikációs tesztekben a tokenek a testHelpers függvényeiből származnak. A refresh token tesztek cookie-n keresztül adják át a tokent, az access tokenes tesztek Authorization headerben küldik.
Az endpointok adatbázisfüggő részeit a mockok vezérlik. Például login esetén a getUserByEmail mock határozza meg, hogy a user nem létezik, tiltott, törölt vagy aktív. Foglalásnál a service, provider, salon és konfliktusellenőrzések mind mockolt válaszokból épülnek fel. Search teszteknél a geokódolás és közeli szalon szűrés szintén mockolt service válaszon keresztül fut.

Helper-ek szerepe a folyamatban

A helper-ek és lokális setup függvények a tesztek stabilitását adják. A setTestEnv() biztosítja, hogy a JWT aláírás és ellenőrzés konzisztens legyen. A token helper-ekkel könnyen szimulálható érvényes customer/provider szerepkörű vagy lejárt tokenes kérés. Az aktív, tiltott vagy törölt fiókállapotot külön adatbázis-mockok adják.
A mock helper-ek csökkentik az ismétlést. Például a calendar tesztekben a provider ellenőrzés két egymást követő adatbázislekérdezését is elő kell készíteni; ezt lokális helper fogja össze. A user tesztekben hasonló minta látható az AuthMiddleware customer státuszellenőrzésére.

Összegzés

A Bookly backend tesztrendszere Jest és Supertest alapú, gyorsan futtatható, mockolt függőségekkel izolált tesztkészlet. A tesztek lefedik az autentikációt, tokenkezelést, jogosultsági middleware-t, customer API-kat, provider API-kat, keresést, naptár- és szolgáltatáskezelést, lokációs service-t és időblokk-kezelést.

A rendszer fő erőssége, hogy az API szerződéseket és üzleti szabályokat ellenőrzi: státuszkódokat, válaszstruktúrákat, token/cookie működést, validációkat, tiltott állapotokat, tranzakciós ágakat és sikeres létrehozási/törlési folyamatokat.

A mockolt adatbázis- és service réteg miatt a tesztek nem függenek élő adatbázistól, e-mail küldéstől, fájlrendszertől vagy külső geokódoló szolgáltatástól. Ez karbantartható és determinisztikus tesztkörnyezetet ad.

A jelenlegi lefedettség backend szempontból erős alapot biztosít a kritikus API útvonalakra és központi service logikákra. Nem teljes rendszerintegrációs teszt élő adatbázissal, viszont jól skálázható: új endpointokhoz további Supertest fájlok, új üzleti logikákhoz pedig külön unit tesztek illeszthetők.

Felhasználói dokumentáció

1. Bevezetés

A Bookly egy szolgáltatásfoglaló webalkalmazás, amelynek célja, hogy a vendégek egyszerűen találjanak megfelelő szalont vagy szolgáltatót, majd online időpontot foglaljanak. A rendszer elsősorban olyan szolgáltatásokhoz készült, ahol az ügyfélfogadás előre egyeztetett időpont alapján történik.
A Bookly központi eleme a szalon. A vendégek a felületen szalonokat böngészhetnek, szolgáltatásokat és munkatársakat tekinthetnek meg, időpontot választhatnak, majd kezelhetik saját foglalásaikat. A rendszer támogatja a szalonok keresését, a szalonadatlapok megtekintését, az online foglalást, a mentett szalonokat, az értékeléseket, a várólistát és az üzenetváltást is.
A szolgáltatók külön felületen kezelhetik a mindennapi munkájukhoz kapcsolódó adatokat. Ide tartozik többek között a naptár, a foglalások, a szolgáltatások, az árak, az időtartamok, a szünetek és egyéb időblokkok kezelése. A szolgáltatói regisztráció során új szalon is létrehozható, illetve meglévő szalonhoz is lehet csatlakozni szalonkóddal. A szalon egyes adatainak és a csapatnak a kezelése jogosultsághoz kötött lehet.
Az admin felület a rendszer felügyeletére szolgál. Az admin áttekintheti és kezelheti a felhasználókat, szolgáltatókat, szalonokat, foglalásokat és értékeléseket, valamint megtekintheti a rendszerhez kapcsolódó statisztikákat és naplókat.
A Bookly három fő felhasználói szerepkört különböztet meg:

- **Vendég (Customer):** szalonokat keres, időpontot foglal, kezeli a saját foglalásait, mentett szalonjait, profilját, értékeléseit és üzeneteit.
- **Szolgáltató (Provider):** a szalon működéséhez kapcsolódó feladatokat kezeli, például a naptárat, foglalásokat, szolgáltatásokat, elérhetőséget, ügyfeleket, munkatársakat és üzeneteket.
- **Admin:** a teljes rendszer működését felügyeli, beleértve a felhasználók, szolgáltatók, szalonok, foglalások, értékelések és naplók kezelését.

Ez a dokumentáció a Bookly használatát mutatja be felhasználói szemszögből. A következő fejezetek röviden összefoglalják a rendszer fő funkcióit, majd külön bemutatják a vendég, a szolgáltató és az admin felület használatát.

2. A rendszer fő funkciói

A Bookly fő funkciói a három felhasználói szerepkör köré épülnek. A vendégek szalonokat kereshetnek és időpontot foglalhatnak, a szolgáltatók a szalon működését és foglalásait kezelhetik, az admin pedig a teljes rendszer felügyeletét látja el.

2.1. Felhasználói fiókok és szerepkörök

A rendszer külön felületet biztosít a vendégeknek, a szolgáltatóknak és az adminnak. A vendégek saját fiókot hozhatnak létre, majd bejelentkezés után elérik a személyes irányítópultjukat. A szolgáltatók külön szolgáltatói regisztrációt használnak, amely során új szalont hozhatnak létre, vagy meglévő szalonhoz csatlakozhatnak szalonkóddal. Az admin külön admin belépési felületen keresztül éri el a rendszerfelügyeleti funkciókat.
A szerepkörök eltérő jogosultságokkal rendelkeznek. A vendég a saját foglalásait és profilját kezeli, a szolgáltató a hozzá tartozó szalonhoz és foglalásokhoz kapcsolódó

adatokat kezeli, az admin pedig rendszerszintű áttekintést és moderálási lehetőségeket kap.

2.2. Vendég funkciók

A vendégek a Bookly felületén szalonokat kereshetnek és böngészhetnek. A keresés működhet szalonnév, szolgáltatástípus, cím, helyszín vagy aktuális földrajzi helyzet alapján. A rendszer térképes megjelenítést, keresési javaslatokat, kiemelt vagy legjobban értékelt szalonokat és ajánlott találatokat is megjeleníthet.
A vendég megnyithatja egy szalon adatlapját, ahol láthatja a szalon alapadatait, címét, nyitvatartását, munkatársait, szolgáltatásait, árait, időtartamait és értékeléseit. A szalon elmenthető a mentett helyek közé, így később könnyebben újra elérhető.
A rendszer egyik központi funkciója az online időpontfoglalás. A vendég a foglalás során munkatársat, szolgáltatást, dátumot és szabad időpontot választ. A foglalás véglegesítése előtt összefoglalót lát, majd a sikeres foglalás megjelenik a saját foglalásai között.
Ha nincs megfelelő szabad időpont, a vendég várólistára is feliratkozhat. A várólistán dátumtartomány és opcionális napszak adható meg. A vendég később megtekintheti és lemondhatja saját várólistás bejegyzéseit.
A vendég a saját felületén kezelheti közelgő és korábbi foglalásait. A közelgő időpontok lemondhatók, a foglalások naptárba is menthetők, a teljesült foglalások után pedig értékelés adható. Az értékelés külön vonatkozhat a szalonra és a szolgáltatóra, és szöveges véleményt is tartalmazhat.
A vendég profiljában kezelheti személyes adatait, profilképét és jelszavát. Az e-mail cím a profil szerkesztésekor nem módosítható. A felület támogatja a saját fiók törlését is. A rendszer 30 napos visszaállítási időszakot jelez; ezalatt a fiók újraaktiválható, a határidő lejárta után pedig már nem állítható vissza.

2.3. Szolgáltatói funkciók

A szolgáltatók saját irányítópultot kapnak, ahol áttekinthetik a működésükhöz kapcsolódó legfontosabb adatokat. A felületen elérhető az áttekintés, a naptár, a szolgáltatások, az elérhetőség, a szalon kezelése, az ügyfelek, a csapat és az üzenetek kezelése.
A szolgáltatói naptárban láthatók a foglalások havi és napi nézetben. A szolgáltató megtekintheti a foglalások részleteit, új időpontot hozhat létre, valamint szükség esetén foglalást törölhet. A rendszer kezeli az időpontütközéseket, és figyelembe veszi a meglévő foglalásokat, szüneteket, időblokkokat és nyitvatartási adatokat.
A szolgáltató kezelheti saját szolgáltatásait. Egy szolgáltatáshoz név, leírás, időtartam, ár, elérhetőségi státusz és képek tartozhatnak. A szolgáltatások létrehozhatók, módosíthatók és törölhetők, ha ezt a rendszer állapota engedi.
Az elérhetőség kezelése során a szolgáltató szüneteket és időblokkokat állíthat be. Ezek lehetnek egyszeri, többnapos vagy ismétlődő időszakok is. A szalon nyitvatartási adatai és arculati elemei, például a logó vagy banner, megfelelő jogosultsággal kezelhetők.
A szolgáltatói felület ügyfélkezelést is biztosít. A szolgáltató láthatja a hozzá kapcsolódó regisztrált és vendég ügyfeleket, megtekintheti foglalási előzményeiket, és bizonyos esetekben emlékeztetőt vagy üzenetet küldhet nekik.
A csapatkezelés menedzseri jogosultsághoz kötött. A menedzser láthatja a szalon munkatársait, módosíthatja státuszukat, kezelheti a menedzseri jogosultságokat, és eltávolíthat munkatársakat a szalonból.

2.4. Admin funkciók

Az admin felület a Bookly rendszer felügyeletére szolgál. Az admin áttekintő statisztikákat láthat a felhasználókról, szolgáltatókról, szalonokról, foglalásokról, bevételekről és új regisztrációkról.

Az admin kezelheti a regisztrált felhasználókat és szolgáltatókat. Mindkét csoportnál elérhető a keresés, szűrés, részletes adatmegtekintés, tiltás, tiltás feloldása és profilkép eltávolítása. Felhasználói fiókoknál GDPR törlés is indítható.
Az admin a szalonokat is felügyelheti. Megtekintheti a szalonok adatait, szolgáltatóit, szolgáltatásait és értékeléseit, módosíthatja a szalon státuszát, valamint eltávolíthat egyes megjelenített tartalmakat, például bannert, logót vagy leírást.
A foglalások admin felületen listázhatók, kereshetők és szűrhetők. Az admin törölhet foglalást, de ehhez törlési indokot kell megadni. Az értékelések megtekinthetők és deaktiválhatók. A rendszer naplói segítségével az admin követheti a fontosabb eseményeket és adminisztratív műveleteket.

2.5. Üzenetek és értesítések

A Bookly üzenetküldési funkciót is tartalmaz. A vendégek és szolgáltatók beszélgetéseket folytathatnak egymással, de az üzenetváltás a rendszer működése alapján foglalási kapcsolathoz kötött. A felület jelzi az olvasatlan üzeneteket, a beszélgetések megnyithatók és törölhetők, a megnyitott beszélgetéseket pedig a rendszer olvasottként kezeli.
A rendszer több helyzetben e-mail értesítést is tud küldeni. Ilyen lehet például a regisztráció utáni üdvözlő e-mail, a foglalás visszaigazolása, a foglalás lemondásáról szóló értesítés, a jelszómódosítás visszaigazolása, a várólistás értesítés felszabaduló időpont esetén, valamint a szolgáltató által küldött ügyfél-emlékeztető.

3. A rendszer használata – Vendég (Customer)

Ez a fejezet azt mutatja be, hogyan használja a Bookly-t egy vendég: hogyan hoz létre fiókot, hogyan keres szalont, hogyan foglal időpontot, hogyan kezeli a foglalásait, értékeléseit, üzeneteit és profilját.

3.1. Regisztráció, bejelentkezés és fiók újraaktiválása

A vendég saját fiókot hozhat létre a Bookly rendszerben. A regisztrációhoz meg kell adni a nevet, az e-mail címet, a jelszót és a jelszó megerősítését. A rendszer ellenőrzi, hogy minden kötelező mező ki van-e töltve, az e-mail cím megfelelő formátumú-e, a név legalább 2 karakter hosszú-e, a jelszó legalább 6 karakteres-e, valamint, hogy a két jelszó megegyezik-e.
Sikeres regisztráció után a rendszer a bejelentkezési oldalra irányítja a vendéget. Bejelentkezni e-mail címmel és jelszóval lehet. Sikeres bejelentkezés után a vendég a saját irányítópultjára kerül.
A vendég fő menüpontjai:

- Áttekintés
- Kiemelt szalonok
- Foglalásaim
- Helyeim
- Üzenetek
- Profil (a felhasználói menüből)

Ha a vendég korábban törlésre jelölte a fiókját, bejelentkezéskor a rendszer újraaktiválási oldalra irányíthatja. Ilyenkor újra meg kell adni a nevet és a telefonszámot; a cím megadása opcionális. Sikeres újraaktiválás után a fiók ismét használható.

3.2. Szalon keresése és böngészése

A vendég az Áttekintés oldalon szalonokat kereshet és böngészhet; a találatokon belül láthatja a szalonhoz tartozó szolgáltatókat is. A keresés történhet szalonnév, szolgáltatástípus, cím vagy település alapján, illetve a böngésző aktuális helyzetének használatával.
A kereső javaslatokat is megjeleníthet, például szalonokra vagy szolgáltatástípusokra. A helyszín mező címjavaslatokat is adhat. Ha a vendég a jelenlegi helyzetét szeretné használni, a böngésző helymeghatározási engedélyt kérhet.
A keresési eredmények kártyákon és térképen jelenhetnek meg. Egy szalonkártyán látható például a szalon neve, címe, értékelése, szolgáltatóinak száma és távolsága, ha a rendszer rendelkezik ehhez szükséges helyadatokkal. Az Áttekintés oldalon ajánlott szalonok és legújabb értékelések is megjelenhetnek.
A Kiemelt szalonok menüpontban a legjobban értékelt szalonok böngészhetők. Itt szolgáltatástípus szerint is lehet szűrni, és további szalonok is betölthetők.

3.3. Szalon adatlapja és mentett helyek

A szalon kártyájáról megnyitható a részletes szalonadatlap. Itt a vendég láthatja a szalon nevét, típusát, nyitvatartási állapotát, nyitvatartási napjait és idejét, címét, telefonszámát, e-mail címét, leírását, munkatársait, szolgáltatásait, árait, időtartamait, értékeléseit és térképes helyszínét.
A szalon adatlapjáról közvetlenül indítható foglalás. Egyes szolgáltatásoknál szolgáltatáskép is megjelenhet. A munkatársak külön is láthatók és a vendég akár konkrét munkatárshoz is indíthat foglalást.
A vendég elmentheti a számára fontos szalonokat. A mentett szalonok a Helyeim menüpontban jelennek meg. Ha egy szalon már nem fontos, ugyanazzal a mentés funkcióval eltávolítható a mentett helyek közül. A szalon mentéséhez bejelentkezés szükséges.

3.4. Időpont foglalása

A foglalás a szalon adatlapjáról indítható. A folyamat lépésenként vezeti végig a vendéget.
A foglalás fő lépései:

1. Munkatárs kiválasztása

   A vendég választhat konkrét munkatársat, vagy használhatja a Bármelyik munkatárs lehetőséget.

2. Szolgáltatás kiválasztása

   A rendszer megjeleníti az elérhető szolgáltatásokat, azok időtartamát és árát.

3. Dátum és időpont kiválasztása

   A vendég naptárból választ napot, majd a rendszer által felkínált szabad időpontok közül választ.

4. Foglalás ellenőrzése, megjegyzés megadása és véglegesítés

   A vendég egy összefoglaló nézetben ellenőrizheti a szalon, munkatárs, szolgáltatás, dátum, időpont és ár adatait. Ugyanitt opcionálisan megjegyzést is írhat a foglaláshoz. A vendég a foglalás véglegesítésével hozhatja létre az időpontfoglalást.

Foglaláshoz bejelentkezés szükséges. A vendég profiljában névnek, e-mail címnek és telefonszámnak is szerepelnie kell. Ha ezek közül valamelyik hiányzik, a rendszer a profil kitöltésére irányítja a vendéget.
A rendszer nem enged múltbeli időpontra foglalni, figyelembe veszi a szalon nyitvatartását, a munkatárs elérhetőségét, a meglévő foglalásokat és a szüneteket. Ha az időpont közben foglalttá válik, a vendégnek másik időpontot kell választania.

3.5. Várólista

Ha a vendég nem talál megfelelő szabad időpontot, feliratkozhat várólistára. Ehhez először ki kell választani a munkatársat és a szolgáltatást, majd az időpontválasztás résznél elérhető a várólista lehetősége.
A várólistán meg kell adni egy dátumtartományt. Opcionálisan választható preferált napszak is: reggel, délután vagy este. A rendszer először ellenőrzi, hogy a megadott időszakban van-e már szabad időpont. Ha talál ilyet, jelzi a vendégnek. Ha nincs megfelelő időpont, létrejön a várólistás bejegyzés.
A vendég a Foglalásaim részen belül láthatja a saját várólistás bejegyzéseit. Itt megtekintheti, hogy van-e szabad időpont a megadott időszakban, és lemondhatja a várólistás bejegyzést. Ha megfelelő időpontot talál, a szalon adatlapjáról új foglalást indíthat. Lemondás esetén a bejegyzés eltűnik a várólistából.

3.6. Foglalások megtekintése és kezelése

A Foglalásaim menüpontban a vendég áttekintheti saját időpontjait. A felület havi naptárnézetet, napi idővonalat, közelgő foglalásokat, korábbi foglalásokat és várólistás bejegyzéseket is megjelenít.
Egy foglalásnál látható a szolgáltatás, a szalon, a szolgáltató, a dátum, az időpont, az időtartam, az ár, az állapot és a megjegyzés. Ha egy foglalást töröltek, a törlés oka is megjelenhet.
Közelgő foglalásnál a vendég:

- megtekintheti a részleteket
- szerkesztheti a megjegyzést
- lemondhatja a foglalást
- naptárba mentheti az időpontot

A naptárba mentésnél Google Calendar és Apple Calendar lehetőség jelenik meg. Fontos, hogy a vendég a foglalás dátumát vagy időpontját közvetlenül nem módosíthatja; ha másik időpontot szeretne, a meglévő foglalást le kell mondania, majd új foglalást kell indítania.
Korábbi, teljesített foglalásoknál a vendég értékelést adhat.

3.7. Értékelések

Értékelést csak teljesített foglalás után lehet adni. Az értékelés során külön kell értékelni a szalont és a szolgáltatót. Mindkettőnél 1 és 5 közötti csillagos értékelés adható. Szöveges vélemény opcionálisan írható a szalonhoz és a szolgáltatóhoz is.
Ha a vendég már adott értékelést egy foglaláshoz, a rendszer betölti a korábbi értékelést, és azt módosítani is lehet. Nem teljesített, lemondott vagy törölt foglalás nem értékelhető.

3.8. Üzenetek

Az Üzenetek menüpontban a vendég beszélgethet szolgáltatókkal. A rendszer működése alapján új beszélgetést csak olyan szolgáltatóval lehet indítani, akinél a vendégnek már

volt vagy van foglalása. Ha még nincs kapcsolódó foglalás, nem jelenik meg választható szolgáltató új beszélgetéshez.
A vendég meglévő beszélgetést megnyithat, üzenetet küldhet, olvashatja az üzenetelőzményeket, és törölheti a beszélgetést. A rendszer jelzi az olvasatlan üzenetek számát, és a megnyitott beszélgetéseket olvasottként kezeli.

3.9. Profil, fiók törlése és kijelentkezés

A vendég a profil menüből nyithatja meg a saját profilját. Itt látható és kezelhető a profilkép, a név, az e-mail cím, a telefonszám, a cím, a fiók státusza és a regisztráció dátuma.
A profilkép JPG, PNG vagy WebP formátumban tölthető fel, legfeljebb 5 MB méretben. A név, telefonszám és cím szerkeszthető. Az e-mail cím a felületen nem módosítható. Ha a vendég címet ad meg, akkor az irányítószámot, várost, utcát és házszámot együtt kell kitölteni.
A jelszó módosításához meg kell adni a jelenlegi jelszót, az új jelszót és az új jelszó megerősítését. Az új jelszónak legalább 6 karakter hosszúnak kell lennie.
A vendég a profiljában kezdeményezheti a fiók törlését. Ehhez meg kell adni a jelszót, valamint megerősítésként be kell írni a „TÖRLÉS” szót. A felület 30 napos visszaállítási időszakot jelez. A törlés érinti a fiókhoz kapcsolódó adatokat, például a mentett szalonokat és az aktív foglalásokat.
Kijelentkezni a profil menüből lehet. Kijelentkezés után a rendszer eltávolítja az aktuális bejelentkezést, és a vendég visszakerül a kezdőoldalra.

4. A rendszer használata – Szolgáltató (Provider)

Ez a fejezet azt mutatja be, hogyan használja a Bookly-t egy szolgáltató. A szolgáltatói felületen a felhasználó kezelheti saját foglalásait, szolgáltatásait, szüneteit, ügyfeleit, üzeneteit és profilját. Ha a szolgáltató menedzseri jogosultsággal rendelkezik, akkor a szalon adatait és a csapatot is kezelheti.

4.1. Regisztráció és bejelentkezés

A szolgáltató kétféleképpen regisztrálhat: új szalon létrehozásával vagy meglévő szalonhoz való csatlakozással. Új szalon létrehozásakor meg kell adni a szalon nevét, címét, típusát és rövid leírását. A címet a rendszer pontos helyadatokkal kezeli, ezért érvényes címet kell kiválasztani.
Meglévő szalonhoz való csatlakozáskor a szolgáltató a szalon meghívókódját adja meg. Ezt a kódot a szalon menedzsere tudja megosztani vele. A szolgáltató személyes adatai minden esetben szükségesek: teljes név, e-mail cím, telefonszám, jelszó és a jelszó megerősítése. A szolgáltatói jelszónak legalább 8 karakter hosszúnak kell lennie.
Sikeres regisztráció után a rendszer a szolgáltatói bejelentkezéshez irányítja a felhasználót. Bejelentkezni e-mail címmel és jelszóval lehet, majd a szolgáltatói irányítópult nyílik meg.

4.2. Szolgáltatói irányítópult és áttekintés

A szolgáltatói irányítópulton a fő menüpontok: Áttekintés, Naptár, Szolgáltatások, Elérhetőség, Szalon kezelés, Ügyfelek, Csapat és Üzenetek. Mobilnézetben néhány menüpont a Több gomb alatt érhető el.
Az Áttekintés oldalon a szolgáltató gyors összefoglalót lát a napi munkáról. Itt jelenik meg a mai foglalások száma, a heti bevétel, az új ügyfelek száma és a következő időpontok listája.

4.3. Naptár, foglalások és manuális időpontok

A Naptár menüpontban a szolgáltató havi naptárat és napi idővonalat lát. A kiválasztott napon megjelennek a foglalások és a szünetek. A naptár jelzi, ha egy napon van foglalás, illetve, ha a nap teljesen betelt.
Egy foglalás részleteinél látható az ügyfél neve, a szolgáltatás, az időpont kezdete és vége, az ügyfél elérhetőségei, az ár, a megjegyzés és a foglalás állapota. Foglalt állapotú időpont esetén a szolgáltató törölheti a foglalást. Regisztrált ügyfél esetén a részletes nézetből üzenetküldés is indítható.
A szolgáltató manuálisan is létrehozhat új foglalást. A foglalás lehet regisztrált felhasználóhoz vagy vendéghez kapcsolódó időpont. Regisztrált felhasználónál létező e-mail címet kell megadni. Vendégfoglalásnál név szükséges, valamint legalább e-mail cím vagy telefonszám. A foglaláshoz ki kell választani a szolgáltatást, a dátumot és az időpontot, megjegyzés pedig opcionálisan adható meg. A rendszer ellenőrzi, hogy az időpont a szalon nyitvatartásán belül van-e, és nem ütközik-e másik foglalással vagy szünettel.

4.4. Szünetek és elérhetőség kezelése

A szolgáltató szüneteket hozhat létre, hogy bizonyos időszakokra ne lehessen hozzá foglalni. Szünet létrehozható egy napra, több napra, valamint ismétlődően naponta vagy hetente. Megadható a dátum vagy dátumtartomány, a kezdő és záró idő, az ismétlődés típusa, az ismétlődés vége és egy megjegyzés.
A szünetek a Naptárban és az Elérhetőség menüpontban is megjelennek. Az Elérhetőség oldalon külön szűrhető az összes, az ismétlődő és az egyszeri szünet. A szünetek szerkeszthetők és törölhetők. Ismétlődő szünetnél a rendszer külön kezeli, hogy csak egy adott alkalomra vagy az összes jövőbeli alkalomra vonatkozzon-e a módosítás. Ha a szünet meglévő foglalással ütközne, a rendszer figyelmeztetést ad.

4.5. Szolgáltatások kezelése

A Szolgáltatások menüpontban a szolgáltató saját szolgáltatásait kezeli. Egy szolgáltatáshoz megadható a név, leírás, időtartam, ár és státusz. A státusz lehet elérhető vagy nem elérhető. Foglalásnál csak az elérhető szolgáltatások használhatók.
A szolgáltatásokhoz referenciaképek is feltölthetők. Egy szolgáltatáshoz legfeljebb 5 kép tartozhat, JPG, PNG vagy WebP formátumban, legfeljebb 5 MB méretben. A szolgáltatás szerkeszthető és törölhető, de ha aktív foglalás kapcsolódik hozzá, a rendszer nem engedi a törlést.

4.6. Szalonadatok, arculat és jogosultságok

A Szalon kezelés menüpontban a szolgáltató megtekintheti a szalon adatait. A szalon adatainak módosításához menedzseri jogosultság szükséges. A menedzser módosíthatja a szalon nevét, címét, telefonszámát, e-mail címét, típusát, nyitvatartási napjait, nyitási és zárási idejét, leírását és státuszát. A státusz lehet nyitva, felújítás alatt vagy zárva.
A szalon megosztási kódja is ezen a felületen látható. Ezzel a kóddal tud új szolgáltató csatlakozni a meglévő szalonhoz. A menedzser az arculatot is kezelheti: feltölthet szalonlogót, bannerképet, választhat bannerszínt, illetve eltávolíthatja a bannerképet. A képfeltöltésnél JPG, PNG és WebP formátum használható, legfeljebb 5 MB méretben.

4.7. Ügyfelek, csapat és üzenetek

Az Ügyfelek menüpontban a szolgáltató azokat az ügyfeleket látja, akikhez foglalás kapcsolódik. A lista név vagy e-mail cím alapján kereshető, és tartalmazhat regisztrált ügyfeleket és vendégfoglalásból származó ügyfeleket is. Az ügyfél részleteinél megjelenik az elérhetőség, a foglalások száma, az összes költés, az első foglalás dátuma, a foglalási előzmények és az értékelés, ha van ilyen. E-mail címmel rendelkező ügyfélnek emlékeztető küldhető, regisztrált ügyfél esetén pedig üzenetküldés is indítható.

A Csapat menüpont csak menedzseri jogosultsággal érhető el. Itt a menedzser látja a szalon munkatársait, azok aktív vagy inaktív állapotát, valamint azt, hogy ki menedzser. A menedzser aktiválhat vagy deaktiválhat munkatársat, menedzseri jogot adhat vagy vehet el, illetve eltávolíthat munkatársat a szalonból. A rendszer védi azt az esetet, hogy a szalon menedzser nélkül maradjon, ezért az utolsó menedzser eltávolítása vagy lefokozása nem engedélyezett. A csapatnézetben egy munkatársat kiválasztva megjelenik az adott munkatárs napi naptára, ahol láthatók a foglalásai és szünetei, és új foglalás is létrehozható számára.
Az Üzenetek menüpontban a szolgáltató beszélgethet regisztrált ügyfelekkel. Megnyithat meglévő beszélgetéseket, új beszélgetést indíthat olyan regisztrált ügyfelekkel, akikhez foglalás kapcsolódik, üzenetet küldhet, láthatja az olvasatlan üzeneteket, és törölhet beszélgetést. Az üzenetek valós időben frissülnek.

4.8. Szolgáltatói profil és kijelentkezés

A szolgáltató a jobb felső profilmenüből szerkesztheti saját profilját. Módosítható a profilkép, a név, a telefonszám és a bemutatkozás. Az e-mail cím megjelenik, de nem szerkeszthető. Profilképként JPG, PNG vagy WebP kép tölthető fel, legfeljebb 5 MB méretben.
A jelszó módosításához meg kell adni a jelenlegi jelszót, az új jelszót és az új jelszó megerősítését. Az új jelszónak legalább 8 karakter hosszúnak kell lennie. A szolgáltató a profilmenüből tud kijelentkezni; kijelentkezés után a szolgáltatói irányítópult csak újabb bejelentkezés után érhető el.

5. Admin funkciók

Az admin felület a Bookly rendszer felügyeletére és moderálására szolgál. Az admin nem vendégként vagy szolgáltatóként használja a rendszert, hanem áttekinti a teljes alkalmazás működését, kezeli a felhasználókat, szolgáltatókat, szalonokat, foglalásokat, értékeléseket és a rendszer naplóit. Az admin funkciók csak admin jogosultsággal érhetők el.

5.1. Bejelentkezés és admin felület

Az admin külön admin bejelentkezési oldalon tud belépni. A bejelentkezéshez e-mail cím és jelszó szükséges. Sikeres bejelentkezés után az admin az admin irányítópultra kerül. Az admin felület oldalsó menüből használható. A fő menüpontok: Áttekintés, Felhasználók, Szolgáltatók, Szalonok, Foglalások, Értékelések és Naplók. Az admin a bal oldali menü alsó részén található Kijelentkezés gombbal tud kilépni.

5.2. Áttekintés

Az Áttekintés oldalon az admin összesített adatokat lát a rendszerről. Megjelenik az aktív felhasználók száma, az aktív szolgáltatók száma, a nyitott szalonok száma, az adott havi foglalások száma, az összes foglalás száma, valamint a teljesült foglalásokból számolt összes bevétel.
Az oldal mutatja az elmúlt 7 nap új felhasználóit és új szolgáltatóit, a foglalásokat állapot szerinti bontásban, a legutóbbi foglalásokat, a legjobban értékelt szalonokat és az elmúlt hónapok bevételi trendjét. Az adatok a Frissítés gombbal újratölthetők.

5.3. Felhasználók kezelése

A Felhasználók menüpontban az admin a rendszer felhasználói fiókjait látja. A lista kereshető név, e-mail cím vagy telefonszám alapján, és státusz szerint is szűrhető. A táblázatban megjelenik a felhasználó neve, e-mail címe, telefonszáma, szerepe, státusza és utolsó belépése.

Egy felhasználóra kattintva részletes nézet nyílik meg. Itt láthatók az alapadatok, például az e-mail cím, telefonszám, cím és regisztráció dátuma, valamint külön fülön a felhasználó foglalásai és értékelései.
Az admin a felhasználóknál elvégezheti a tiltást és a tiltás feloldását, eltávolíthatja a profilképet, valamint GDPR törlést indíthat. A GDPR törlés a személyes adatok anonimizálását jelenti, és a rendszer ezt külön megerősítéshez köti. A felhasználó részletes nézetében az admin a kapcsolódó foglalást indoklással törölt állapotba teheti, illetve a kapcsolódó értékelést deaktiválhatja.

5.4. Szolgáltatók kezelése

A Szolgáltatók menüpontban az admin a szolgáltatói fiókokat tekintheti át. A lista kereshető név, e-mail cím vagy szalon alapján, és státusz szerint is szűrhető. A táblázatban látható a szolgáltató neve, e-mail címe, szalonja, szerepe és státusza.
Egy szolgáltató részletes nézetében megjelennek az alapadatok, a kapcsolódó szalon, a szolgáltató foglalásai, értékelései és szolgáltatásai. Az admin letilthatja a szolgáltatót, feloldhatja a tiltást, illetve eltávolíthatja a szolgáltató profilképét. A részletes nézetben a szolgáltatóhoz tartozó foglalás indoklással törölhető, a hozzá kapcsolódó értékelés pedig deaktiválható.

5.5. Szalonok kezelése

A Szalonok menüpontban az admin a rendszerben szereplő szalonokat látja. A lista kereshető szalonnév, cím vagy típus alapján, és státusz szerint is szűrhető. A szalonoknál megjelenik a név, cím, típus, szolgáltatók száma, átlagos értékelés és státusz.
Az admin módosíthatja a szalon státuszát: nyitva, felújítás alatt vagy zárva. Egy szalon részletes nézetében láthatók a szalon alapadatai, a hozzá tartozó szolgáltatók, szolgáltatások és értékelések. Moderálási célból az admin eltávolíthatja a szalon bannerképét, logóját vagy leírását. A szalonhoz tartozó értékelések szintén deaktiválhatók.

5.6. Foglalások és értékelések moderálása

A Foglalások menüpontban az admin az összes rendszerbeli foglalást áttekintheti. A lista kereshető ügyfél, szolgáltató, szalon vagy szolgáltatás alapján, és szűrhető foglalási státusz szerint. A foglalásoknál látható az időpont, az ügyfél, a szolgáltató, a szalon, a szolgáltatás, az ár és az állapot.
A felület összesítést is mutat az összes, foglalt, teljesült és lemondott foglalásokról. Az admin törölhet foglalást, de ehhez kötelező törlési indokot megadni. A törlés nem egyszerű eltávolítást jelent: a foglalás törölt állapotba kerül, az indok pedig a rendszerben megmarad.
Az Értékelések menüpontban az admin a vendégek által adott értékeléseket látja. A lista kereshető felhasználó, szalon, szolgáltató vagy komment alapján, és szűrhető aktív vagy inaktív állapot szerint. Az admin látja a szalonra és szolgáltatóra adott csillagos értékelést, a dátumot és a szöveges megjegyzéseket. Az értékelések nem szerkeszthetők át, hanem deaktiválhatók.

5.7. Rendszernaplók

A Naplók menüpontban az admin a rendszer fontos eseményeit tudja áttekinteni. A naplók szűrhetők eseményszint szerint, például INFO, WARN vagy CRITICAL szintre, valamint művelettípus alapján.
A naplóbejegyzésekben megjelenik az esemény időpontja, szintje, műveletneve, az indító szereplő, a célobjektum és a részletes leírás. A naplók között szerepelhetnek például bejelentkezések, regisztrációk, tiltások, GDPR-törlések, foglalástörlések, értékelés

deaktiválások, szalonstátusz-módosítások és egyéb moderációs vagy rendszeresemények.